Lightweight Tensorized Neural Networks for Hyperspectral Image Classification

Tian-Yu Ma, Heng-Chao Li, *Senior Member, IEEE*, Rui Wang, Qian Du, *Fellow, IEEE*, Xiuping Jia, *Fellow, IEEE*, and Antonio Plaza, *Fellow, IEEE*

***Abstract*— Deep learning methods have demonstrated excellent performance in hyperspectral image (HSI) classification. However, these methods mainly focus on improving the classification accuracy while ignoring their high complexity. By considering that the data formats of both HSIs and network weights can be represented in the form of tensors, we develop a new lightweight tensorized neural network (TNN) for HSI classification that takes advantage of low-rank tensor decomposition techniques to reduce complexity. First, inspired by tensor train (TT)-based tensorized convolutional layers, a new tensorized 2D convolutional layer based on chain calculation (with better expression ability) is introduced. Based on this innovation, a new lightweight 2D-TNN is designed for HSI classification. Furthermore, to better preserve the intrinsic structure of HSI data, a new lightweight 3D-TNN is proposed by extending the tensorized 2D convolutional layers to their 3D versions. Quantitative and comparative experiments on three widely used datasets show that the proposed models are able to achieve the state-of-the-art performance (with a low number of model parameters) for different training sample sizes, especially for very small training sets.**

***Index Terms*— Classification, convolutional neural networks (CNNs), hyperspectral images (HSIs), network compression, tensor decomposition.**

I. Introduction

HYPERSPECTRAL images (HSIs) can be expressed as a three-order tensor to simultaneously describe the spatial and spectral information of land cover classes. These images comprise high spectral resolution in the form of hundreds of contiguous wavelength channels, ranging from visible to near-infrared wavelengths. This allows to perform classification of materials on the surface of the Earth with subtle spectral differences [1], [2]. Specifically, HSI classification has been used in plenty of applications, such as precision agriculture [3], forest health analysis [4], water quality analysis [5], geological exploration [6], and urban planning [7], yielding satisfactory results.

A. Supervised Classification Methods

Supervised methods have received significant attention for HSI classification. Traditional methods include support vector machines (SVMs) [8], joint sparse representation classifier (JSRC) [9], random forest (RF) [10], and multinomial logistic regression (MLR) [11]. Based on the aforementioned methods, several improvement works have been presented, such as composite kernel SVM [12], correntropy-based robust JSRC [13], and so on [14], [15]. Despite their widespread use, these methods may suffer from the handcrafted features, whose design requires the prior information, thus having low classification performance in complex scenarios.

B. Deep Learning-Based Classification Methods

Due to the ability of deep learning methods to automatically extract features with high-level semantics information, these methods have reached excellent results in image classification [16], object detection [17], [18], and fault detection [19]. Besides, they have also been exploited in the field of HSI classification [20], [21]. Chen et al. [22] proposed three different stacked autoencoder (SAE) models, using spectral, spatial, and spatial–spectral information, to generate deep features. Meanwhile, deep belief networks (DBNs), as a variant of the AE model, were also applied to HSI classification [23]. However, these models can only process samples arranged in the form of 1-D vectors, disregarding the rich spatial information present in HSIs. Convolutional neural networks (CNNs), unlike the above methods, can overcome this limitation. Makantasis et al. [24] employed a 2D-CNN to automatically extract the spatial–spectral features from HSIs and conducted the classification task with a multilayer perceptron. Hu et al. [20] and Chen et al. [25] presented three different CNN models (i.e., 1D-CNN, 2D-CNN, and 3D-CNN) for spectral, spatial, and spatial–spectral classification of HSI data. Li et al. [26] compared the SAE, DBN, 2D-CNN, and 3D-CNN, demonstrating that 3D-CNN-based methods can produce superior results. As deep learning advances, some novel deep models, such as residual neural networks

Manuscript received 16 June 2022; revised 8 October 2022; accepted 29 October 2022. Date of publication 28 November 2022; date of current version 13 December 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 62271418, Grant 61871335, and Grant 61801404; and in part by the Fundamental Research Funds for the Central Universities under Grant 2682020ZT35. *(Corresponding author: Heng-Chao Li.)*

Tian-Yu Ma and Rui Wang are with the School of Information Science and Technology, Southwest Jiaotong University, Chengdu 611756, China.

Heng-Chao Li is with the School of Information Science and Technology and the National Engineering Laboratory of Integrated Transportation Big Data Application Technology, Southwest Jiaotong University, Chengdu 611756, China (e-mail: lihengchao_78@163.com).

Qian Du is with the Department of Electrical and Computer Engineering, Mississippi State University, Starkville, MS 39762 USA.

Xiuping Jia is with the School of Engineering and Information Technology, University of New South Wales, Canberra, ACT 2612, Australia.

Antonio Plaza is with the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, Escuela Politécnica, University of Extremadura, 10071 Cáceres, Spain.

Digital Object Identifier 10.1109/TGRS.2022.3225438

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

(ResNets) [27], densely connected networks (DenseNets) [28], and capsule networks (CapsNets) [29], have been proposed as extensions of CNNs to extract more descriptive high-level semantic features for improving the classification performance. Zhong et al. [30] followed the work presented in [27] and developed a spatial–spectral ResNet for HSI classification. Wang et al. [31] suggested using the DenseNet to extract multiscale spatial–spectral features to improve the classification performance. In the light of [29], Paoletti et al. [32] proposed a spatial–spectral CapsNet that outperforms the traditional CNN and ResNet models. Lately, there have been some novel networks combined with CNNs for HSI classification. For example, Yang et al. [33] introduced a hyperspectral image transformer (HiT) network by embedding convolution operations into the transformer structure to capture the subtle spectral discrepancies and to convey the local spatial information. Liu et al. [34] proposed a CNN-enhanced graph convolutional network (CEGCN) by fusing their complementary spatial–spectral features at pixel and superpixel levels and alleviating the structural incompatibility. Moreover, considering high spectral correlation in HSI bands, the recurrent neural network (RNN) [35] and its variations, such as long short-term memory (LSTM) [36] and convolutional LSTM (ConvLSTM) [37], [38], were introduced for feature extraction and classification of HSIs. Mou et al. [39] adopted the RNN and LSTM models for HSI classification. Liu et al. [40] presented a bidirectional-ConvLSTM to extract spatial–spectral features. This method fails, however, to fully utilize the ability of ConvLSTM for long-range dependency modeling. Furthermore, Hu et al. [41] designed spatial–spectral ConvLSTM neural networks to solve the aforementioned problem. In this context, a spatial–spectral ConvLSTM 2D neural network (SSCL2DNN) and its extension (SSCL3DNN) were designed for HSI classification. Although deep models have shown excellent performance in HSI classification, they are frequently overparameterized, which increases the demands for computing and storage resources (as well as the risk of overfitting), thus limiting their practical applications [42].

C. Reducing the Complexity of Deep Learning-Based Methods With Model Compression

Model compression can decrease the complexity of deep models and overcome the aforementioned problems [43]. The main methods for this purpose include compact architecture design [44], model pruning [45], weight quantization or sharing [46], knowledge distillation [47], and low-rank matrix/tensor decomposition [48], among others [49]. At present, there have been some works in the field of HSI classification aimed at obtaining low-complexity deep models by using model compression methods. Among them, compact structure design has been one of the most widely used. In this context, Fang et al. [50] designed a squeeze convolution module (SCM) to replace the traditional convolutional layer. Building on [50], Meng et al. [51] developed a lightweight spatial–spectral convolution module (LS^2CM) to further reduce the complexity of the model by introducing depthwise separable convolutions. With the idea of [44] in mind, Cui et al. [52] proposed a lightweight and efficient spatial attention module to extract spatial features for classification of HSIs. Li and Duan [53] designed a lightweight network framework (LiteDenseNet) by reducing the computational cost and the number of parameters through a group convolution. Based on [53], Cui et al. [54] proposed LiteDepthwiseNet, in which 3D depthwise separable convolutions replace group convolutions to further reduce the complexity of the network. As a follow-up, the LiteSCANet framework was presented in [55]. This framework introduces the attention mechanism (along with iterative pruning) to the model while designing a light residual unit to enable the network to achieve higher classification performance and lower model complexity. Furthermore, Liu et al. [56] employed a Siamese CNN that contains two subnetworks with weight sharing for HSI classification. This means that there are much fewer parameters to train, decreasing the risk of overfitting. Inspired by knowledge distillation, Cao et al. [57] proposed a compressed neural network-based HSI classification method that uses a large teacher network to guide the training of a small student network, thereby achieving similar performance as the teacher network under the premise of low complexity. In [58], neural structure search was used to automatically find optimal network structures for different HSI datasets. Roy et al. [59] designed a hybrid spectral CNN (HybridSN) by combining 2D- and 3D-CNNs, which also reduces the complexity of the model. Among the aforementioned model compression methods, compact structure design provides limited compression rates (CRs) and often requires an elaborate design, model pruning needs pretraining of the corresponding uncompressed deep model (and some methods may obtain irregular sparse weight tensors with poor applicability to hardware), weight sharing generally lacks sufficient CR, and knowledge distillation involves the processing of two networks, resulting in low training efficiency [42].

D. Alternative: Low-Rank Matrix/Tensor Decomposition

In contrast to the above methods, low-rank matrix/tensor decomposition has the support of linear algebraic theory, allowing to sufficiently compress a well-trained model from scratch. Meanwhile, compressed low-rank convolution kernels can directly replace the standard convolution kernels in existing CNN models to decrease their complexity, which can make use of many existing standard network structures and reduce the cost of network design. Novikov et al. [60] proposed the use of fully connected (FC) layers in a tensor train (TT) format, preserving the original expression ability of the layers while drastically decreasing the number of parameters. Kossaifi et al. [61] designed tensor contraction layer and tensor regression layer as the end-to-end trainable components of neural network to achieve the compact structure. In [62], the 2D convolution kernel was reconstructed as a higher order tensor that was then compressed into the TT format. Su et al. [63] introduced tensorized convolutional layers based on chain computation. Recently, Wang et al. [64] applied TT decomposition [65] to the compression of 3D convolution kernels. Meanwhile, compressed convolutional

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

and FC layers have also been widely used in fields related to medical image classification [66], brainprint recognition [67], video summarization [68], and video scene segmentation [69]. Nevertheless, to the best of our knowledge, there exists minimal work using low-rank matrix/tensor decomposition to obtain low-complexity models for HSI classification. Makantasis et al. [70] presented tensor-based linear and nonlinear models for HSI classification, in which the rank-1 decomposition is utilized to reduce the number of parameters, especially showing better performance with small training sets. Hu et al. [71] applied TT decomposition to the compression of the SSCL2DNN model and developed a lightweight SSTTCL2DNN model, but it suffered from performance loss due to its complex structure within the ConvLSTM layer and the reduced number of parameters. To compensate for the performance loss, a tensor attention residual block module was presented to recover the accuracy of SSTTCL2DNN through the attention mechanism. However, Hu et al. [71] achieved the compression only for the convolution kernels, while a large number of parameters in the FC layers were not considered.

E. Contributions of This Work

In this article, two lightweight tensorized neural network (TNN) models, containing tensorized convolutional and FC layers, are proposed for HSI spatial and spatial–spectral classification, respectively. Inspired by [62], new tensorized convolutional layers based on chain calculation and their 3D versions are designed, with which new lightweight 2D-TNN and 3D-TNN models are proposed for HSI classification with a small number of required parameters. The main contributions of this article can be summarized as follows.

1) To reduce the complexity of deep models in HSI classification, a new lightweight convolutional structure, i.e., 2DConv-TR-C, is designed based on the tensorized 2D convolutional layer with chain computation (namely, 2DConv-TT-C) [62] and tensor ring (TR) decomposition. Using these modules, a new lightweight deep model, i.e., 2D-TNN, is proposed for the spatial feature extraction and classification of HSIs.
2) To better utilize the spectral information of HSIs, the 3D lightweight structures (namely, 3DConv-TT-C and 3DConv-TR-C) are further developed based on the above 2D tensorized convolutional layers. As a result, a novel deep model (namely 3D-TNN) is constructed for spatial–spectral classification of HSI data. Due to the large amount of redundancy contained in the 3D-CNN, our 3D-TNN achieves higher CRs.

The rest of this article is structured as follows. Section II introduces a graphical representation of tensor, tensor operation, and tensor decomposition, as well as the tensorized convolutional layers and networks. The proposed lightweight deep models for spatial and spatial–spectral classification of HSIs are, respectively, described in Sections III and IV. Section V provides a comprehensive quantitative and comparative analysis of the proposed models. Section VI concludes this article with some remarks and hints at plausible future research lines.

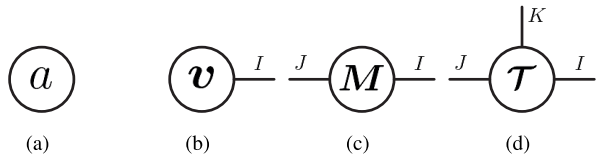


Fig. 1. Tensor diagrams. (a) Scalar. (b) Vector. (c) Matrix. (d) Tensor.

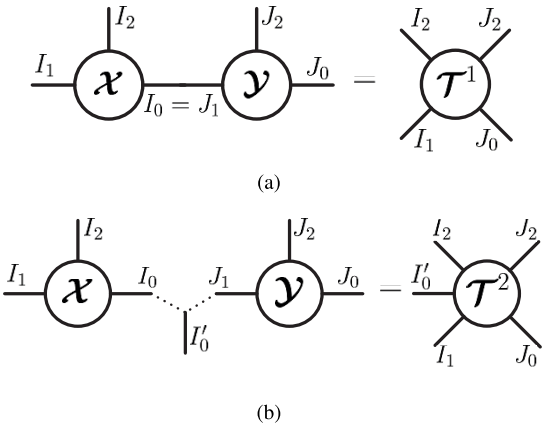


Fig. 2. Tensor diagrams of tensor operations. (a) Mode-(0, 1) tensor contraction. (b) Mode-(0, 1) tensor convolution.

II. Background on Tensor Algebra and Notations

In this section, tensor notations and operations, as well as tensor decompositions and their application, are briefly presented as the related background.

A. Tensor

A multidimensional array can be defined as a high-order tensor. Formally, an N-order tensor is represented as $\mathcal{W} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$, where $I_n (1 \leq n \leq N)$ is the dimensionality of the nth mode of $\mathcal{W}$.

1) Tensor Diagrams: Tensor diagrams, which are graphical representations of multidimensional arrays, are used to visualize the tensors and tensor operations. The diagram of a tensor is drawn as a node with several edges. The number of edges extending from the node represents its order, where each edge corresponds to one mode. As shown in Fig. 1, a scalar $a \in \mathbb{R}$ is a zero-order tensor, so it has no edges. A vector $\boldsymbol{v} \in \mathbb{R}^I$ that is a one-order tensor has only one edge, a matrix $\boldsymbol{M} \in \mathbb{R}^{I \times J}$ has two edges, which is a two-order tensor, and a three-order tensor $\mathcal{T} \in \mathbb{R}^{I \times J \times K}$ has three edges. In tensor operations, these graphical representations can clearly describe the relationship between different tensors.

2) Tensor Operations: As presented in Fig. 2, there are two basic tensor operations. The shape of the line linking the nodes indicates the type of operation: the solid line represents the tensor contraction and the dashed line denotes the tensor convolution. The tensor contraction and the tensor convolution operations are defined as

$$\begin{aligned} \mathcal{T}^1 &= \mathcal{X} \times_1^0 \mathcal{Y} \\ \mathcal{T}^2 &= \mathcal{X} *_1^0 \mathcal{Y} \end{aligned} \tag{1}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

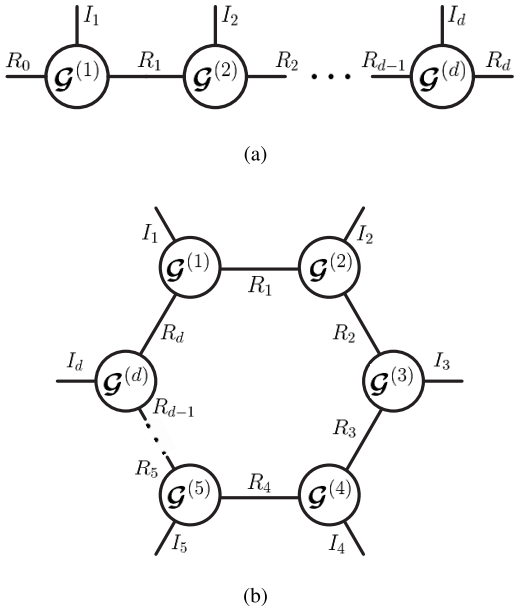


Fig. 3. Diagrams of tensor decomposition. (a) TT decomposition. (b) TR decomposition.

where $\mathcal{X} \in \mathbb{R}^{I_0 \times I_1 \times I_2}$ and $\mathcal{Y} \in \mathbb{R}^{J_0 \times J_1 \times J_2}$ are the input tensors, $\mathcal{T}^1 \in \mathbb{R}^{I_1 \times I_2 \times J_0 \times J_2}$ and $\mathcal{T}^2 \in \mathbb{R}^{I_0' \times I_1 \times I_2 \times J_0 \times J_2}$ are the corresponding output tensors, and $I_0 = J_1$. In the following, each operator is denoted with subscript and superscript to indicate the mode of the operations. For example, $\times_1^0$ is mode-$(0, 1)$ tensor contraction. The generalized tensor operations in Fig. 2 are based on three-order tensors, which can also be extended to higher order tensors [63].

B. Tensor Decomposition

1) TT Decomposition: The tensor diagram of TT decomposition is shown in Fig. 3(a). TT decomposition factorizes a d-order tensor $\mathcal{T}^3 \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_d}$ into several three-order tensors $\{\mathcal{G}^{(l)}\}_{l=1}^{d}$, and its formula can be expressed as

$$\mathcal{T}^3 = \mathcal{G}^{(1)} \times_0^{-1} \mathcal{G}^{(2)} \times_0^{-1} \cdots \times_0^{-1} \mathcal{G}^{(d)} \tag{2}$$

where $\mathcal{G}^{(l)} \in \mathbb{R}^{R_{l-1} \times I_l \times R_l}$, $\forall l \in [1, d]$. $\{R_n\}_{n=0}^{d}$ are the ranks, which are called TT-ranks for short and affect the complexity of TT decomposition. The values of R_0 and R_d are fixed to 1. Specifically, there are some methods to find the stable quasi-optimal rank in TT decomposition, such as truncated singular value decomposition.

2) TR Decomposition: Based on TT decomposition, we set R_0 and R_d to R $(R > 1)$ so that $\mathcal{G}^{(1)}$ and $\mathcal{G}^{(d)}$ are connected to obtain the TR decomposition. This decomposition form is shown as a ring-like structure and can be given by

$$\mathcal{T}^3 = \left(\mathcal{G}^{(1)} \times_0^{-1} \cdots \times_0^{-1} \mathcal{G}^{(d-1)}\right)\left(\times_{-1}^{0} \circ \times_0^{-1}\right)\mathcal{G}^{(d)} \tag{3}$$

where $\mathcal{G}^{(l)} \in \mathbb{R}^{R_{l-1} \times I_l \times R_l}$, $\forall l \in [1, d]$. $\{R_n\}_{n=0}^{d}$ are TR-ranks and $R_n > 1$. In addition, TT-ranks always have a fixed form, i.e., $R_0 = R_d = 1$, which may limit the expressiveness of TT decomposition, but this limitation can be overcome due to the ring form of the TR decomposition [72].

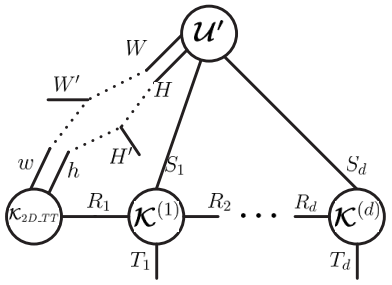


Fig. 4. Tensor diagram of 2DConv-TT-C.

C. Tensorized Convolutional Layer and Network

In [62], the 2D convolution kernel, which is a four-order tensor $\mathcal{K} \in \mathbb{R}^{w \times h \times S \times T}$, was reshaped into a higher order tensor $\mathcal{K}_{2D} \in \mathbb{R}^{w \times h \times S_1 \times \cdots \times S_d \times T_1 \times \cdots \times T_d}$ that is decomposed in the TT format according to [60]. The reconstructed convolution kernel is factorized into several tensors, which can be expressed as

$$\mathcal{K}_{2D} = \mathcal{K}_{2D_\mathrm{TT}} \times_0^{-1} \mathcal{K}^{(1)} \times_0^{-1} \cdots \times_0^{-1} \mathcal{K}^{(d)} \tag{4}$$

where $\mathcal{K}_{2D_\mathrm{TT}} \in \mathbb{R}^{w \times h \times R_1}$, $\mathcal{K}^{(l)} \in \mathbb{R}^{R_l \times S_l \times T_l \times R_{l+1}}$, $\forall l \in [1, d)$, and $\mathcal{K}^{(d)} \in \mathbb{R}^{R_d \times S_d \times T_d}$ are factor tensors. $\{R_n\}_{n=1}^{d}$ are TT-ranks. w and h are the width/height of kernel, respectively. S and T are the numbers of input and output channels, while $S = \prod_{l=1}^{d} S_l$ and $T = \prod_{l=1}^{d} T_l$. At the same time, the chain calculation form of tensorized convolutional layer, namely, 2DConv-TT-C, was also realized, as shown in Fig. 4.

TNNs consist of tensorized convolutional and FC layers, which are obtained by using various types of tensor decomposition (i.e., CANDECOMP/PARAFAC (CP), Tucker, TT, and TR) as projection methods to compress traditional layers. Among them, both TT and TR decomposition belong to the tensor network decomposition techniques, which can avoid the curse of dimensionality [64]. Furthermore, TT decomposition is widely used because of its simplicity of expression, while its general form (TR decomposition) can overcome its limitations. Thus, in this article, TT and TR decomposition are selected to compress the weight tensor of the convolutional and FC layers to achieve effective model compression for HSI classification.

III. 2D-TNN

After Hu et al. [20] introduced the 1D-CNN for the HSI spectral classification, more improved CNN-based models have been proposed, and they have achieved unprecedented accuracy [25], [26], [30], [31], [32]. However, they require deeper and wider architectures, with a large number of model parameters and higher computational complexity [57].

In order to overcome the aforementioned problem, inspired by [62] and based on TR decomposition theory, a novel tensorized convolutional layer based on chain calculation is proposed, which is designated 2DConv-TR-C. Then, using 2DConv-TT-C and 2DConv-TR-C, two new TNNs (i.e., 2D-TNN-TT and 2D-TNN-TR) are designed for spatial feature extraction and classification of HSIs.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

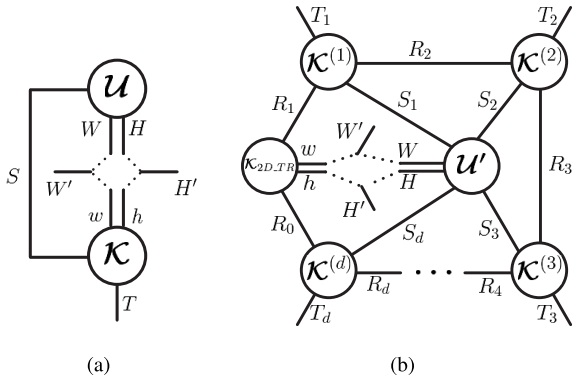


Fig. 5. Tensor diagrams of convolutional layers. (a) Traditional convolutional layer. (b) 2DConv-TR-C.

A. Design of 2DConv-TR-C

Convolutional layers, nonlinear activation layers, pooling layers, FC layers, and classification layers are all basic components of CNNs. Among them, the number of parameters is mainly concentrated in the convolutional and FC layers, but they can be compressed. In a 2D convolutional layer, the four-order kernel $\mathcal{K}$ maps the three-order input tensor $\mathcal{U} \in \mathbb{R}^{W\times H\times S}$ to another three-order output tensor $\mathcal{V} \in \mathbb{R}^{W'\times H'\times T}$. The tensor diagram of the convolutional layer is shown in Fig. 5(a), whose formula is defined as

$$\mathcal{V} = \mathcal{U}\left(*_0^0 \circ *_1^1 \circ \times_2^2\right)\mathcal{K}. \tag{5}$$

The tensorized convolutional layer has a different format, whose input tensor needs to be reshaped as $\mathcal{U}' \in \mathbb{R}^{W\times H\times S_1\times\cdots\times S_d}$. Similarly, the output tensor and the convolution kernel are reshaped as $\mathcal{V}' \in \mathbb{R}^{W'\times H'\times T_1\times\cdots\times T_d}$ and $\mathcal{K}_{2D}$, respectively.

Algorithm 1 2DConv-TR-C

Input: Input tensor $\mathcal{U} \in \mathbb{R}^{W\times H\times S}$;
Weight $\mathcal{K}_{2D}$ of 2DConv-TR-C defined in (6).
Output: Output tensor $\mathcal{V} \in \mathbb{R}^{W'\times H'\times T}$.

$\mathcal{U} \in \mathbb{R}^{S\times W\times H\times 1} \leftarrow$ reshape $\mathcal{U}$
$\mathcal{K}'_{2D_TR} \in \mathbb{R}^{w\times h\times R_0R_1} \leftarrow$ reshape $\mathcal{K}_{2D_TR}$
$\mathcal{U}^{(0)} \in \mathbb{R}^{S\times W'\times H'\times R_0R_1} \leftarrow \mathcal{U}(*_0^1 \circ *_1^2 \circ \times_{-1}^{-1})\mathcal{K}'_{2D_TR}$
$\mathcal{U}^{(0)} \in \mathbb{R}^{R_0\times W'\times H'\times S_1\times\cdots\times S_d\times R_1} \leftarrow$ reshape $\mathcal{U}^{(0)}$
for $i = 1 \longrightarrow d-1$ **do**
  $\mathcal{U}^{(i)} \in \mathbb{R}^{R_0\times W'\times H'\times S_{i+1}\times\cdots\times S_d\times T_1\cdots\times T_i\times R_{i+1}} \leftarrow \mathcal{U}^{(i-1)}(\times_1^3 \circ \times_0^{-1})\mathcal{K}^{(i)}$
end for
$\mathcal{U}^{(d)} \in \mathbb{R}^{W'\times H'\times T_1\times\cdots\times T_d} \leftarrow \mathcal{U}^{(d-1)}(\times_{-1}^0 \circ \times_1^3 \circ \times_0^{-1})\mathcal{K}^{(d)}$
$\mathcal{V} \in \mathbb{R}^{W'\times H'\times T} \leftarrow$ reshape $\mathcal{U}^{(d)}$
return $\mathcal{V}$.

According to Section II-C and TR decomposition theory, a new tensorized convolutional layer based on chain calculation is proposed, named 2DConv-TR-C. The reconstructed convolution kernel $\mathcal{K}_{2D}$ in 2DConv-TR-C is rewritten as

$$\mathcal{K}_{2D} = \left(\mathcal{K}_{2D_TR} \times_0^{-1} \cdots \times_0^{-1} \mathcal{K}^{(d-1)}\right)\left(\times_{-1}^0 \circ \times_0^{-1}\right)\mathcal{K}^{(d)} \tag{6}$$

where $\mathcal{K}^{(l)} \in \mathbb{R}^{R_l\times S_l\times T_l\times R_{l+1}}$, $\forall l \in [1, d]$, and $\mathcal{K}_{2D_TR} \in \mathbb{R}^{R_0\times w\times h\times R_1}$ are factor tensors. $\{R_n\}_{n=0}^{d+1}$ are TR-ranks and $R_0 = R_{d+1}$. The tensor diagram of 2DConv-TR-C is shown in Fig. 5(b). Different from 2DConv-TT-C, all cores in 2DConv-TR-C are four-order tensors and they form a ring-like structure. The corresponding CR from the traditional convolution kernel $\mathcal{K}$ to the compressed kernel can be mathematically expressed as

$$\mathrm{CR} = \frac{whST}{whR_0R_1 + \sum_{l=1}^{d} S_lT_lR_lR_{l+1}}. \tag{7}$$

It can be seen that the CR depends heavily on the choice of ranks under the premise of determining the kernel size. A higher rank leads to a higher complexity of 2DConv-TR-C and, conversely, a lower rank leads to a smaller complexity.

The detailed chain calculation process of 2DConv-TR-C is described in Algorithm 1. The input feature $\mathcal{U}$ first performs a 2D convolution operation with $\mathcal{K}_{2D_\mathrm{TR}}$, and the output result $\mathcal{U}^{(0)}$ performs a tensor contraction operation with $\{\mathcal{K}^{(l)}\}_{l=1}^{d}$ successively, to obtain the final output feature.

B. Design of 2D-TNN

To reduce the large number of parameters needed by the CNN for HSI spatial classification, the modules 2DConv-TT-C and 2DConv-TR-C are, respectively, utilized to construct two different 2D-TNNs for spatial feature extraction and classification of HSIs, as well as model compression. According to the used modules, these two 2D-TNNs are named 2D-TNN-TT and 2D-TNN-TR. The structure of the proposed 2D-TNNs is shown in Fig. 6.

First, the samples containing HSI spatial information are extracted from the original data. Principal component analysis (PCA) is used to reduce the spectral dimension and the first principal component is retained. After that, for each pixel in the first PC, the data in a local spatial window of size $s \times s \times 1$ are cropped as a sample and used as the input to 2D-TNNs.

The proposed 2D-TNNs consist of three tensorized 2D convolutional layers and a tensorized FC layer. Each tensorized 2D convolutional layer can be 2DConv-TT-C or 2DConv-TR-C. Since the size of the input sample $\mathcal{F}_0$ is $s \times s \times 1$, the input channel of the first layer is 1, which enables the first tensorized 2D convolutional layer to degenerate into a traditional convolutional layer (2DConv). Its mathematical expression is

$$\mathcal{F}_1 = \mathrm{ReLU}(BN(\mathcal{F}_0 *_{2D} \mathcal{K}_1)) \tag{8}$$

where $\mathcal{F}_0$ and $\mathcal{F}_1$ are the input and output of the first layer, respectively, $\mathcal{K}_1$ is the weight tensor, BN stands for batch normalization layers, ReLU is the activation function, and $*_{2D}$ refers to the 2D convolution operation. To obtain a better compression effect, the input and output channels of the second and third layers are decomposed as much as possible. For example, the tensor format of the second layer in 2D-TNN is $5 \times 5 \times (4 \times 8) \times (8 \times 8)$, and the one of the third layer is $4 \times 4 \times (4 \times 4) \times (4 \times 8) \times (4 \times 4)$. It should be noted that the first two numbers in tensor format are the size of convolution kernel, while the first and second numbers in the parentheses

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

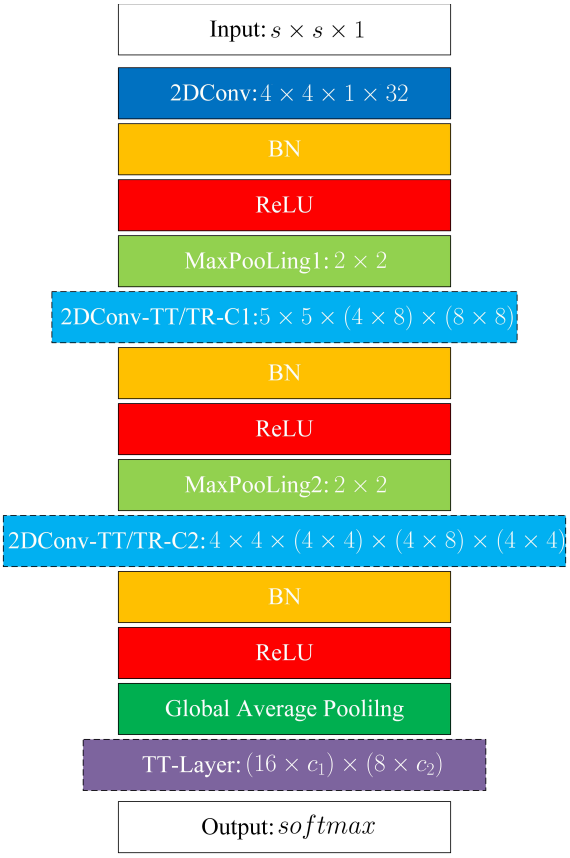


Fig. 6. Structure of the proposed 2D-TNN.

represent the input and output channels, respectively, that is, the output channels of three layers are 32, 64, and 128. These can be expressed as

$$\mathcal{F}_2 = \mathrm{ReLU}\big(BN\big(\mathcal{F}_1 *_{2D} \mathcal{K}_2^0 \times \mathcal{K}_2^1 \times \mathcal{K}_2^2\big)\big) \tag{9}$$

$$\mathcal{F}_3 = \mathrm{ReLU}\big(BN\big(\mathcal{F}_2 *_{2D} \mathcal{K}_3^0 \times \mathcal{K}_3^1 \times \mathcal{K}_3^2 \times \mathcal{K}_3^3\big)\big) \tag{10}$$

where $\mathcal{F}_2$ and $\mathcal{F}_3$ are the outputs of the second and third layers, respectively, and $\mathcal{K}_m^l$ represents the lth kernel of the mth layer.

In addition, following the ideas of [44], each tensorized convolutional layer is followed by a batch normalization (BN) layer and a ReLU activation layer, while the maximum pooling (MaxPooLing) layer is used to reduce the size of features. Meanwhile, considering that the FC layer after feature flattening is one of the main reasons for the high complexity of CNNs and, even with the tensorised FC layer, there will still be a large number of parameters involved, so a global average pooling (GAP) layer is used in the 2D-TNN.

The output feature $\mathcal{F}_3$ of the third tensorized convolutional layer passes through a GAP layer and, then, a feature vector f_{GAP} with a length of 128 is obtained, which can be expressed as

$$f_{GAP} = \mathrm{GAP}(\mathcal{F}_3). \tag{11}$$

Then, by feeding f_{GAP} into the tensorized FC layer, the final output feature will be obtained. Specifically, the TT-layer [60] is uniformly used in different networks, whose tensor format is $(16 \times c_1) \times (8 \times c_2)$ with $c_1 \times c_2$ being the number of classes of the HSI dataset. Finally, the output of the TT-layer passes through a softmax layer to predict the classification result.

IV. 3D-TNN

Considering the characteristics of joint spatial–spectral classification of HSIs and the fact that these images can be expressed as a three-order tensor, tensorized 2D convolutional layers are extended to their 3D versions, which are called 3DConv-TT-C and 3DConv-TR-C in this section, to take full advantage of abundant spectral information. With novel tensorized 3D convolutional layers, a novel lightweight deep model (namely, 3D-TNN) with two TNNs is proposed for spatial–spectral feature extraction and classification of HSIs.

Algorithm 2 3DConv-TT-C

Input: Input tensor $\mathcal{U} \in \mathbb{R}^{W \times H \times D \times S}$;
Weight $\mathcal{K}_{3D}$ of 3DConv-TT-C defined in (8).
Output: Output tensor $\mathcal{V} \in \mathbb{R}^{W' \times H' \times D' \times T}$.

$\mathcal{U} \in \mathbb{R}^{S \times W \times H \times D \times 1} \leftarrow$ reshape $\mathcal{U}$
$\mathcal{U}^{(0)} \in \mathbb{R}^{S \times W' \times H' \times D' \times R_1} \leftarrow \mathcal{U}(*_0^1 \circ *_1^2 \circ *_2^3 \circ \times_{-1}^{-1})\mathcal{K}_{3D_TT}$
$\mathcal{U}^{(0)} \in \mathbb{R}^{W' \times H' \times D' \times S_1 \times \cdots \times S_d \times R_1} \leftarrow$ reshape $\mathcal{U}^{(0)}$
for $i = 1 \longrightarrow d$ **do**
  $\mathcal{U}^{(i)} \in \mathbb{R}^{W' \times H' \times D' \times S_{i+1} \cdots \times S_d \times T_1 \cdots \times T_i \times R_{i+1}} \leftarrow \mathcal{U}^{(i-1)}(\times_1^3 \circ \times_0^{-1})\mathcal{K}^{(i)}$
end for
$\mathcal{V} \in \mathbb{R}^{W' \times H' \times D' \times T} \leftarrow$ reshape $\mathcal{U}^{(d)}$
return $\mathcal{V}$.

A. Design of 3DConv-TT/TR-C

In the traditional 3D convolutional layer, the four-order kernel $\mathcal{K}$ in the 2D convolutional layer is expanded to a five-order one $\mathcal{K}' \in \mathbb{R}^{w \times h \times p \times S \times T}$, where p denotes the depth of the kernel, while its input tensor $\mathcal{U} \in \mathbb{R}^{W \times H \times D \times S}$ and output tensor $\mathcal{V} \in \mathbb{R}^{W \times H \times D \times T}$ are generally four-order tensors. As described in Section III, $\mathcal{K}'$ is also reshaped as a tensorized 3D convolution kernel $\mathcal{K}_{3D} \in \mathbb{R}^{w \times h \times p \times S_1 \times \cdots \times S_d \times T_1 \times \cdots \times T_d}$.

Algorithm 3 3DConv-TR-C

Input: Input tensor $\mathcal{U} \in \mathbb{R}^{W \times H \times D \times S}$;
Weight $\mathcal{K}_{3D}$ of 3DConv-TR-C defined in (9).
Output: Output tensor $\mathcal{V} \in \mathbb{R}^{W \times H \times D \times T}$.

$\mathcal{U} \in \mathbb{R}^{S \times W \times H \times D \times 1} \leftarrow$ reshape $\mathcal{U}$
$\mathcal{K}'_{3D_TR} \in \mathbb{R}^{w \times h \times p \times R_0 R_1} \leftarrow$ reshape $\mathcal{K}_{3D_TR}$
$\mathcal{U}^{(0)} \in \mathbb{R}^{S \times W' \times H' \times D' \times R_0 R_1} \leftarrow \mathcal{U}(*_0^1 \circ *_1^2 \circ *_2^3 \circ \times_{-1}^{-1})\mathcal{K}'_{3D_TR}$
$\mathcal{U}^{(0)} \in \mathbb{R}^{R_0 \times W' \times H' \times D' \times S_1 \times \cdots \times S_d \times R_1} \leftarrow$ reshape $\mathcal{U}^{(1)}$
for $i = 1 \longrightarrow d - 1$ **do**
  $\mathcal{U}^{(i)} \in \mathbb{R}^{R_0 \times W' \times H' \times D' \times S_{i+1} \cdots \times S_d \times T_1 \cdots \times T_i \times R_{i+1}} \leftarrow \mathcal{U}^{(i-1)}(\times_1^3 \circ \times_0^{-1})\mathcal{K}^{(i)}$
end for
$\mathcal{U}^{(d)} \in \mathbb{R}^{W' \times H' \times D' \times T_1 \times \cdots \times T_d} \leftarrow \mathcal{U}^{(d-1)}(\times_{-1}^0 \circ \times_1^3 \circ \times_0^{-1})\mathcal{K}^{(d)}$
$\mathcal{V} \in \mathbb{R}^{W' \times H' \times D' \times T} \leftarrow$ reshape $\mathcal{U}^{(d)}$
return $\mathcal{V}$.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

The calculations of both 3DConv-TT-C and 3DConv-TR-C are similar to those in the tensorized 2D convolutional layers. The difference lies in the kernel used for the convolution operation. The 3DConv-TT-C is defined as

$$\mathcal{K}_{3D} = \mathcal{K}_{3D_TT} \times_0^{-1} \mathcal{K}^{(1)} \times_0^{-1} \cdots \times_0^{-1} \mathcal{K}^{(d)} \tag{12}$$

where $\mathcal{K}^{(l)} \in \mathbb{R}^{R_l \times S_l \times T_l \times R_{l+1}}$, $\forall l \in [1, d)$, $\mathcal{K}_{3D_TT} \in \mathbb{R}^{w \times h \times p \times R_1}$, and $\mathcal{K}^{(d)} \in \mathbb{R}^{R_d \times S_d \times T_d}$. $\{R_n\}_{n=1}^{d}$ are TT-ranks. Furthermore, 3DConv-TR-C is defined as

$$\mathcal{K}_{3D} = \left(\mathcal{K}_{3D_TR} \times_0^{-1} \cdots \times_0^{-1} \mathcal{K}^{(d-1)}\right)\left(\times_{-1}^{0} \circ \times_0^{-1}\right)\mathcal{K}^{(d)} \tag{13}$$

where $\mathcal{K}^{(l)} \in \mathbb{R}^{R_l \times S_l \times T_l \times R_{l+1}}$, $\forall l \in [1, d]$, and $\mathcal{K}_{3D_TR} \in \mathbb{R}^{R_0 \times w \times h \times p \times R_1}$. $\{R_n\}_{n=0}^{d+1}$ are TR-ranks and $R_0 = R_{d+1}$. The cores, including $\mathcal{K}_{3D_TT}$ and $\mathcal{K}_{3D_TR}$, are utilized to implement the 3D convolution operation to extract spatial–spectral features from HSIs.

In [63], the input feature first performs the tensor contraction operations successively, and its output performs a 3D convolution operation to obtain the final feature. In order to extract the spatial–spectral features from HSIs, when this algorithm is applied to the tensorized 3D convolutional layer, a complex 4-D convolution operation will be required. This disadvantage is overcome in our proposed 3DConv-TT/TR-C by adjusting the order of operations. Specifically, the input feature $\mathcal{U}$ first performs a 3D convolution operation, and the output result $\mathcal{U}^{(0)}$ conducts tensor contraction with $\{\mathcal{K}^{(l)}\}_{l=1}^{d}$ successively to obtain the final output feature. The detailed calculations for 3DConv-TT-C and 3DConv-TR-C are given in Algorithms 2 and 3, respectively.

B. Design of 3D-TNN

Based on the tensorized 3D convolutional layers above, the 2D-TNN in Section III is further extended to a novel deep model (3D-TNN) to utilize spatial and spectral information. Similarly, depending on the module used, our two 3D-TNNs are referred to as 3D-TNN-TT and 3D-TNN-TR. Fig. 7 shows the structure of our newly proposed 3D-TNN.

To reduce the complexity of the whole model, PCA is also used to decrease the spectral dimensionality of HSIs by taking the first K PCs, and the size of the input samples becomes $s \times s \times K \times 1$. The tensor formats of the input and output channels of each layer are consistent with the 2D-TNN in Section III, but the sizes of the convolution kernel become $4 \times 4 \times 4$ or $5 \times 5 \times 5$. Correspondingly, as shown in Fig. 8, the convolution operations in (9) and (10) also become 3D convolution operations. Meanwhile, the kernel size of the MaxPooLing layer is also adjusted to $2 \times 2 \times 2$. The output features of the third tensorized 3D convolutional layer pass through a GAP layer, a TT-layer, and a softmax layer. Then, the final classification result is obtained.

The TT and TR decomposition imposes an implicit low-rank regularization on the network, which helps the CNN to achieve better classification performance at low complexity. In addition, both 2D-TNN and 3D-TNN exhibit low model complexity, which will facilitate the practical deployment of deep models in scenarios with limited hardware resources (such as airborne or spaceborne).

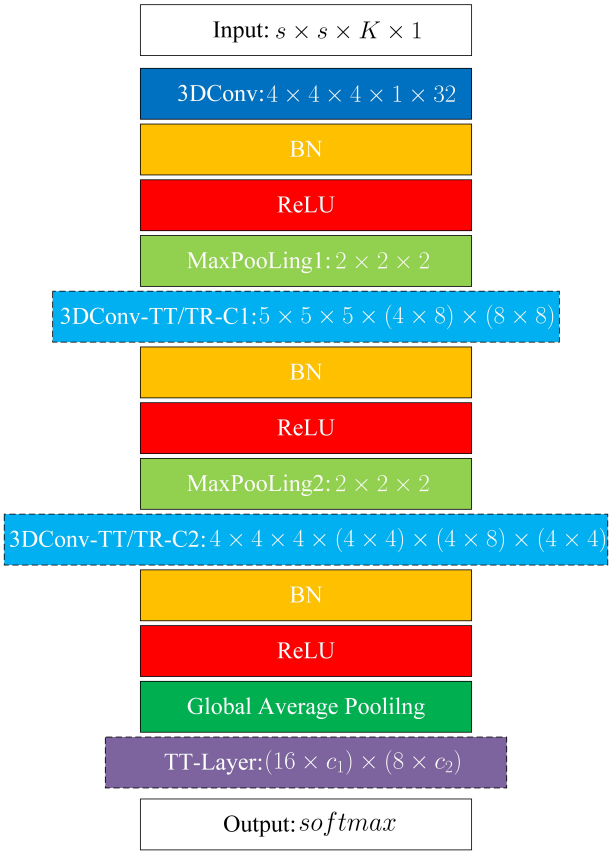


Fig. 7. Structure of the proposed 3D-TNN.

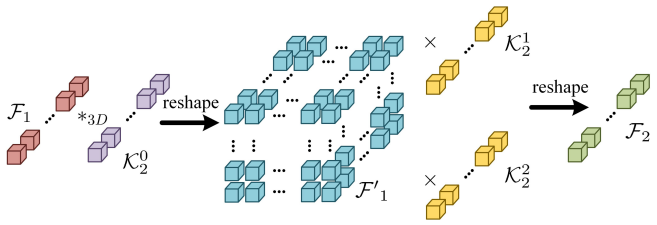


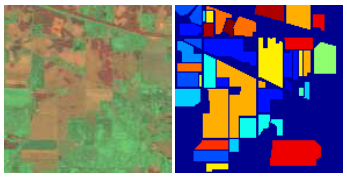
Fig. 8. Second layer of the proposed 3D-TNN, where $*_{3D}$ refers to the 3D convolution operation. $\mathcal{F}_1$ is a four-order tensor, while $\mathcal{F}'_1$ is a six-order one.

V. Experimental Results

In order to quantitatively evaluate the classification performance of the proposed models (2D-TNN and 3D-TNN), we compare them with other approaches such as SVM [73], 2D-CNN [74], 3D-CNN [74], F-3DCNN [75], HybridSN [59], and SSTTCL2DNN [71]. Among them, SVM, 2D-CNN, and our proposed 2D-TNNs are used to evaluate the spatial classification performance, whereas 3D-CNN, F-3DCNN, HybridSN, SSTTCL2DNN, and our proposed 3D-TNNs are compared in the spatial–spectral classification performance. As quantitative evaluation indicators, overall accuracy (OA), average accuracy (AA), and Kappa coefficient (κ), are adopted. Each experiment is conducted ten times to eliminate bias caused by the random selection of the sample and the average value of each evaluation indicator is presented.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

Indian Pines

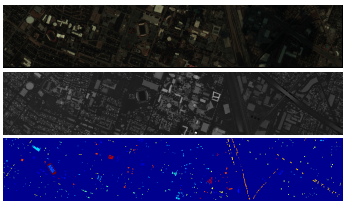


(a)

NO.	Color	Class	Training	Total
1		Alfalfa	5	46
2		Corn-notill	143	1428
3		Corn-mintill	83	830
4		Corn	24	237
5		Grass-pasture	48	483
6		Grass-trees	73	730
7		Grass-pasture-mowed	3	28
8		Hay-windrowed	48	478
9		Oats	2	20
10		Soybean-notill	97	972
11		Soybean-mintill	246	2455
12		Soybean-clean	59	593
13		Wheat	21	205
14		Woods	127	1265
15		Buildings-Grass-Trees-Drives	39	386
16		Stone-Steel-Towers	9	93
Total			1027	10249

(d)

Houston



(b)

NO.	Color	Class	Training	Total
1		Health grass	198	1251
2		Stressed grass	190	1254
3		Synthetic grass	192	697
4		Tress	188	1244
5		Soil	186	1242
6		Water	182	325
7		Residential	196	1268
8		Commercial	191	1244
9		Road	193	1252
10		Highway	191	1227
11		Railway	181	1235
12		Parking lot 1	192	1233
13		Parking lot 2	184	469
14		Tennis court	181	428
15		Running track	187	660
Total			2832	15029

(e)

University of Pavia



(c)

NO.	Color	Class	Training	Total
1		Asphalt	66	6631
2		Meadows	186	18649
3		Gravel	21	2099
4		Trees	31	3064
5		Painted metal sheets	13	1345
6		Bare Soil	50	5029
7		Bitumen	14	1329
8		Self-Blocking Bricks	37	3682
9		Shadows	10	946
Total			428	42774

(f)

Fig. 9. False-color maps and ground-truth maps of three HSI datasets considered in experiments. (a) Indian Pines (bands 20, 40, and 60). (b) Houston (bands 57, 27, and 17). (c) University of Pavia (bands 47, 27, and 13). (d)–(f) Number of training samples of each dataset.

Regarding the hardware system configuration, all experiments are carried out on a desktop computer with an Intel Core i7-10700K CPU, 32 GB of DDR4 RAM with serial speed of 2400 MHz, and an Nvidia GeForce RTX 3080 GPU. In terms of software system configuration, the main programming environments used for implementations are Anaconda 3.5, Pytorch 1.7.0, and Python 3.7.0. Data preparations conducted in experiments, such as PCA, normalization, and sample selection, are realized using MATLAB.

A. HSI Datasets

In our experiments, we used three HSI datasets: Indian Pines, Houston, and University of Pavia.

1) Indian Pines: The dataset was captured in Northwestern Indiana via the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) sensor. The size of the dataset is $145 \times 145 \times 224$ pixels with a spatial resolution of 20 m/pixel and spectral coverage spanning from 0.4 to 2.5 μm. Low-quality bands are removed and the remaining 200 bands are used for classification. After removing the background pixels, a total of 10 249 pixels (including 16 different class labels) are used as samples.

2) Houston: The dataset was acquired by the Compact Airborne Spectrographic Imager (CASI) sensor over the University of Houston and the surrounding area in 2012. It comprises $349 \times 995 \times 144$ pixels with a spatial resolution of 2.5 m/pixel and a spectral coverage of 0.38–1.05 μm. There are 54 129 pixels in total, and after removing the background pixels, a total of 15 029 pixels with a discrete distribution are used as samples, including 15 distinct class labels. The data are available at the IEEE Geoscience and Remote Sensing Society (GRSS) Data and Algorithm Standards Evaluation (DASE) website: http://dase.grss-ieee.org/.

3) University of Pavia: The dataset was captured at the University of Pavia, northern Italy, via the Reflective Optics System Imaging Spectrometer (ROSIS) sensor in 2001. It comprises $610 \times 340 \times 103$ pixels with a spatial resolution of 1.3 m/pixel and a spectral coverage of 0.43–0.86 μm. The dataset contains 42 776 samples divided into nine urban land-cover class labels.

In what follows, all samples for each HSI dataset are divided into training and testing sets. Specifically, 10% and 1% samples of every available labeled class for the Indian Pines and University of Pavia datasets are randomly selected for training, respectively, while the remianing ones are for testing. As far as the Houston dataset is concerned, the choice of 2832 samples for training and 12 197 samples for testing is consistent with the setting of the 2013 IEEE GRSS Data Fusion contest. The false-color maps, ground-truth maps, and number of training samples per class are shown in Fig. 9.

B. Experimental Settings

For all comparison models, the hyperparameters of SVM, 2D-CNN, 3D-CNN, F-3DCNN, HybridSN, and SSTTCL2DNN are set according to [59], [71], [73], [74], and [75] to ensure the optimal performance. To reduce the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

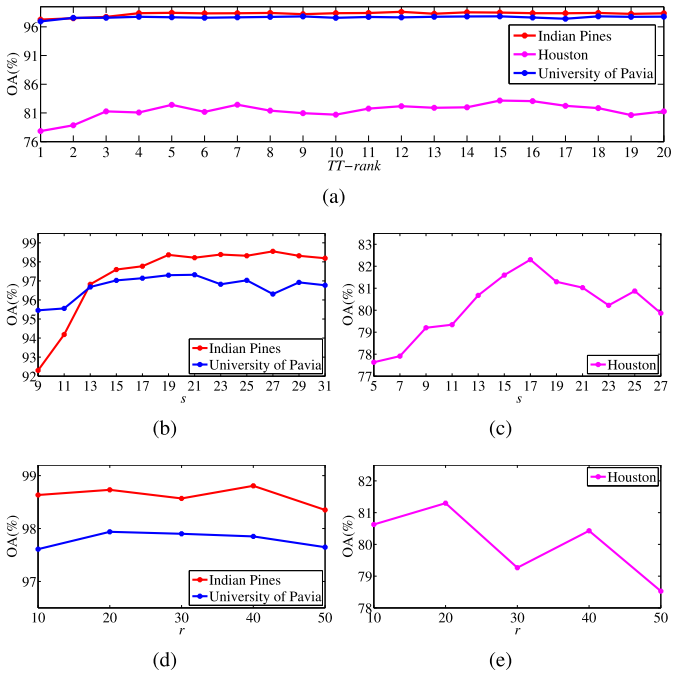


Fig. 10. OA of the proposed 3D-TNN-TT model with different parameters. (a) TT-rank of the tensorized convolutional layers. (b) and (c) Size s of the local window. (d) and (e) Rank r of the TT-layer.

complexity, PCA is used as the method of data dimensionality reduction. In particular, for the 2D-CNN in [74], the first PC is used for the performance analysis of HSI spatial classification. Furthermore, considering that the 2D-CNN is similar in structure to the proposed 2D-TNN models, it is extended to a 3D form for a fair comparison and the first ten PCs are used for performance analysis of all 3D models.

For our proposed models, some critical parameters must be determined, i.e., the ranks (TT-rank and TR-rank) of the tensorized convolutional layers, the size (s) of the local window, the rank (r) of the TT-layer, the size (k) of the convolution kernels, and the number (K) of the PCs.

First, the effect of TT-rank on the accuracy of the proposed 3D-TNN-TT model is given in Fig. 10(a). It can be seen that the OA curves for the Indian Pines and University of Pavia datasets tend to be stable with the variation of TT-rank, while, for the Houston dataset, the OA curve first increases and then shows some fluctuations but reaches its optimum at TT-rank $=$ 15. As such, by comprehensively considering the classification performance in the three datasets, the TT-rank in 3D-TNN-TT is uniformly set to 15, and the ones in 2D-TNN-TT are similarly chosen to simplify the parameter setting. Moreover, for a fair comparison, the TR-rank of the second and third layers in the TR-based models is determined by guaranteeing that the CR of the same layer is analogous. Then, we search s from $\{5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31\}$, r from $\{10, 20, 30, 40, 50\}$, and k from $\{3, 4, 5\}$ by fixing the other two of them, and all experiments are conducted in 3D-TNN-TT. According to the experimental results in Fig. 10, the window sizes (i.e., s) for the Indian Pines, Houston, and University of Pavia datasets are chosen as 27×27, 17×17, and 21×21, and the quasi-optimal values of r are 40, 20, and 20, respectively. In the Indian Pines and University of Pavia datasets, the convolution kernel sizes are set to 4, 5, and 4 for three tensorized convolutional layers. Moreover, to achieve the good performance, the value of k for all tensorized convolutional layers of the proposed models in the Houston dataset is unified to 3. As for K, it is chosen from $\{5, 10, 15, 20\}$. The OA of 3D-TNN-TT as K varies is given in Table I and the quasi-optimal value of K is 10 for three experimental datasets.

TABLE I

CLASSIFICATION PERFORMANCE (OA) OF 3D-TNN-TT WITH DIFFERENT NUMBERS (K) OF THE PCS

K	Indian Pines	Houston	University of Pavia
5	98.85	77.06	97.28
10	**98.91**	**81.87**	**97.89**
15	98.88	80.52	97.81
20	98.68	81.53	97.20

TABLE II

PARAMETER SETTINGS OF THE PROPOSED 2D-TNN

Layer Name	Output Shape	Kernel Size	TT/TR-rank
Input	$27 \times 27 \times 1$	-	-
2DConv	$26 \times 26 \times 32$	4	-
MaxPooling	$13 \times 13 \times 32$	2	-
2D-TT/TR-C	$13 \times 13 \times 64$	5	15/8
MaxPooling	$6 \times 6 \times 64$	2	-
2D-TT/TR-C	$5 \times 5 \times 128$	4	15/12
GAP	128	-	-
TT-Layer	c	-	40
Output	c	-	-

TABLE III

PARAMETER SETTINGS OF THE PROPOSED 3D-TNN

Layer Name	Output Shape	Kernel Size	TT/TR-rank
Input	$10 \times 27 \times 27 \times 1$	-	-
3DConv	$9 \times 26 \times 26 \times 32$	4	-
MaxPooling3D	$4 \times 13 \times 13 \times 32$	2	-
3D-TT/TR-C	$4 \times 13 \times 13 \times 64$	5	15/7
MaxPooling3D	$2 \times 6 \times 6 \times 64$	2	-
3D-TT/TR-C	$1 \times 5 \times 5 \times 128$	4	15/10
GAP	128	-	-
TT-Layer	c	-	40
Output	c	-	-

To ensure the convergence, the training epochs of all proposed models are fixed to 400. The initial learning rate is set to 0.01, and then, it falls by 10% at epochs 200 and 300. For better performance, the initial learning rate is adjusted to 0.001 in the Houston dataset, while the learning rate strategy remains unchanged. The summary of the detailed parameter settings is given in Tables II and III.

C. Classification Results

According to the above experimental settings, the quantitative evaluation results of various deep models are presented in Tables IV–VI and Figs. 11–13. They reveal that the proposed 2D-TNN and 3D-TNN models exhibit advantages such as low model parameters and high classification performance.

In the 2D-CNN, there are three convolutional layers and an FC layer, which helps extract good spatial features, but

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
CLASSIFICATION RESULTS FOR THE INDIAN PINES DATASET USING 10% OF THE AVAILABLE LABELED SAMPLES

Class	SVM	2D-CNN	2D-TNN-TT	2D-TNN-TR	3D-CNN	F-3DCNN	HybridSN	SSTTCL2DNN	3D-TNN-TT	3D-TNN-TR
1	70.73	76.83	**92.68**	91.22	93.41	94.88	93.41	**98.37**	94.63	94.15
2	89.32	83.63	97.34	**97.92**	97.39	92.14	94.74	97.02	**98.04**	98.03
3	91.93	87.18	**98.53**	98.47	99.01	93.47	99.00	96.74	**99.05**	98.98
4	86.50	88.26	**98.77**	98.12	97.51	76.81	95.07	95.62	97.61	**98.45**
5	90.57	88.74	96.59	**97.22**	97.98	96.99	**98.76**	96.32	97.63	97.40
6	97.72	96.62	98.94	**99.21**	99.07	99.12	99.00	98.83	99.12	**99.38**
7	58.00	76.00	**99.00**	92.80	86.80	93.60	**99.60**	92.00	98.40	98.40
8	98.72	99.74	**99.95**	99.90	**99.98**	99.91	99.81	98.84	99.81	**99.98**
9	44.44	60.56	85.56	**91.11**	86.11	80.00	91.11	81.48	91.11	**96.11**
10	77.20	85.09	97.07	**97.17**	96.03	91.27	**98.99**	95.62	97.50	97.74
11	95.02	94.95	99.08	**99.14**	**99.19**	94.37	98.99	98.43	99.01	99.11
12	84.27	86.12	**96.81**	96.69	95.54	85.64	96.70	**96.82**	96.23	96.29
13	89.13	**99.13**	98.26	98.04	98.26	**98.97**	96.76	91.67	97.23	97.46
14	97.41	99.03	**99.91**	99.89	**99.91**	97.75	99.75	98.18	99.78	99.79
15	93.73	96.54	99.36	**99.60**	98.50	92.54	98.16	97.98	**98.93**	98.70
16	68.75	88.69	93.69	**94.52**	95.00	92.02	90.24	92.46	95.23	**95.48**
OA	91.20	91.58	98.37	**98.49**	98.27	93.89	98.10	97.33	98.50	**98.59**
AA	83.34	87.94	96.91	**96.93**	96.23	92.47	96.88	95.40	97.46	**97.84**
κ	89.91	90.38	98.14	**98.28**	98.02	93.03	97.84	96.96	98.29	**98.39**

TABLE V
CLASSIFICATION RESULTS FOR THE HOUSTON DATASET USING THE FIXED SET OF TRAINING SAMPLES

Class	SVM	2D-CNN	2D-TNN-TT	2D-TNN-TR	3D-CNN	F-3DCNN	HybridSN	SSTTCL2DNN	3D-TNN-TT	3D-TNN-TR
1	47.58	**68.74**	64.93	64.36	80.17	**80.67**	80.15	71.61	79.83	79.49
2	44.55	53.64	55.05	**56.23**	78.84	82.64	80.45	79.45	82.24	**82.81**
3	26.19	37.01	42.00	**42.56**	57.40	**78.65**	73.06	64.40	67.51	66.33
4	34.85	64.88	64.06	**66.77**	**88.93**	86.81	88.72	73.13	85.67	86.39
5	53.22	53.38	**61.83**	64.40	96.62	99.91	**99.60**	98.38	96.59	98.01
6	44.06	72.73	74.19	**77.41**	84.83	86.22	80.35	76.36	**93.85**	92.10
7	45.15	**77.30**	75.49	76.69	**85.32**	71.43	74.97	72.79	82.01	80.83
8	16.14	**38.25**	35.92	36.05	58.98	68.69	63.62	**80.59**	73.08	72.88
9	50.24	68.27	68.57	**69.19**	85.19	71.84	72.68	78.59	84.29	**85.63**
10	22.39	43.06	**44.71**	44.64	46.35	55.48	57.75	58.41	59.94	**61.49**
11	32.83	59.27	**63.76**	62.83	67.47	69.65	68.38	65.09	**83.75**	81.89
12	21.88	47.02	51.86	**53.22**	91.18	78.13	87.19	70.36	**95.11**	95.03
13	82.46	**85.75**	78.70	78.59	86.81	65.19	61.23	75.82	88.63	**89.51**
14	48.99	**95.87**	90.85	93.20	90.45	**92.51**	86.03	91.98	84.29	84.49
15	3.81	9.47	14.88	**20.57**	70.76	**92.16**	92.09	71.31	77.02	85.45
OA	36.62	56.37	57.59	**58.63**	77.38	77.42	77.60	74.69	81.79	**82.24**
AA	38.29	58.31	59.12	**60.45**	77.95	78.67	77.75	75.22	82.25	**82.82**
κ	32.08	52.85	54.20	**55.32**	75.54	75.57	75.79	72.52	80.28	**80.77**

TABLE VI
CLASSIFICATION RESULTS FOR THE UNIVERSITY OF PAVIA DATASET USING 1% OF THE AVAILABLE LABELED SAMPLES

Class	SVM	2D-CNN	2D-TNN-TT	2D-TNN-TR	3D-CNN	F-3DCNN	HybridSN	SSTTCL2DNN	3D-TNN-TT	3D-TNN-TR
1	46.58	81.00	90.06	**90.44**	96.11	91.54	92.66	88.67	96.45	**96.61**
2	95.62	97.90	98.23	**98.34**	99.74	97.90	99.17	97.40	99.81	**99.83**
3	36.17	52.45	**75.75**	74.68	81.41	59.47	84.72	71.88	**94.59**	93.86
4	75.85	89.40	89.51	**89.98**	**95.93**	90.33	85.49	83.16	94.37	94.22
5	97.03	91.03	96.68	**97.13**	**99.35**	98.91	97.42	88.84	**99.35**	99.24
6	32.87	66.59	**90.08**	88.99	94.32	94.74	**99.55**	80.10	97.91	98.38
7	22.45	31.29	66.16	**67.60**	82.76	92.31	91.72	72.67	93.81	**97.07**
8	68.86	88.33	92.85	**93.42**	93.93	88.13	88.49	85.27	**97.59**	97.33
9	77.76	85.82	82.61	**85.99**	**95.02**	88.93	74.22	76.05	89.38	90.85
OA	71.37	85.38	92.43	**92.57**	96.22	92.93	94.76	89.18	97.79	**97.95**
AA	61.46	75.98	86.88	**87.39**	93.17	89.14	90.38	82.67	95.92	**96.38**
κ	59.85	80.10	89.91	**90.09**	94.97	90.62	93.04	85.55	97.08	**97.28**

the high complexity of the convolutional layers is significant. Therefore, based on the low-rank TT and TR decomposition, two tensorized 2D convolutional layers based on chain computation are implemented to replace the ordinary convolutional layers. Together with the TT-layer, they form the 2D-TNNs to reduce the number of parameters. On the one hand, compared with 2D-CNN, our proposed 2D-TNNs become narrower and deeper, which provides better global semantics and more abstract detail features. On the other hand, tensor decomposition applies low-rank regularization to the model weights, which reduces noise in the data and leads to the extraction of more expressive features. They both help to improve the accuracy of our models. Among the HSI spatial models, the proposed 2D-TNN-TR reaches the best classification performance with the smallest number of parameters in all three datasets. More specifically, 2D-TNN-TR achieved higher classification performance than 2D-TNN-TT with a lower number of parameters due to its better expression ability, while, compared with 2D-CNN, 2D-TNN-TR improved OA by 6.91%, 2.26%, and 7.19% on the Indian

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

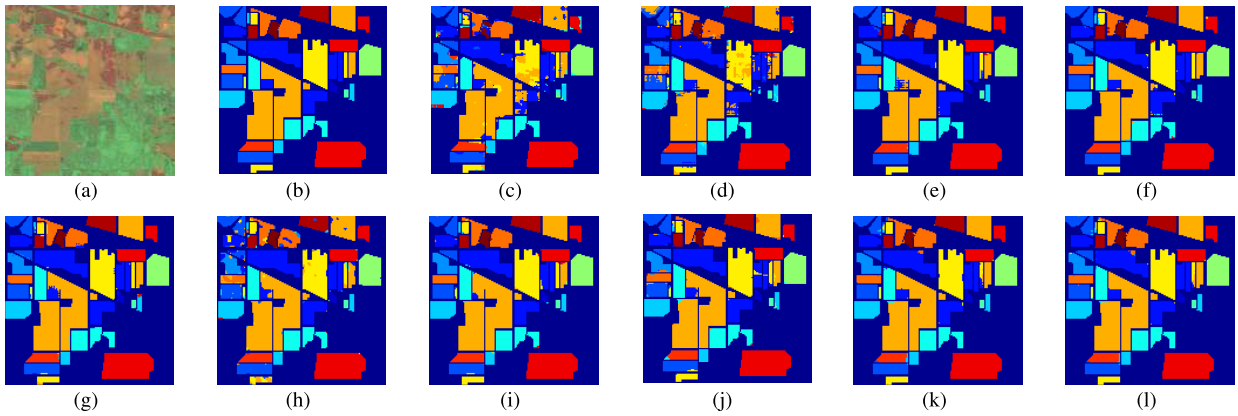


Fig. 11. Classification maps for the Indian Pines dataset. (a) False-color map. (b) Ground-truth map. (c) SVM. (d) 2D-CNN. (e) 2D-TNN-TT. (f) 2D-TNN-TR. (g) 3D-CNN. (h) F-3DCNN. (i) HybridSN. (j) SSTTCL2DNN. (k) 3D-TNN-TT. (l) 3D-TNN-TR.

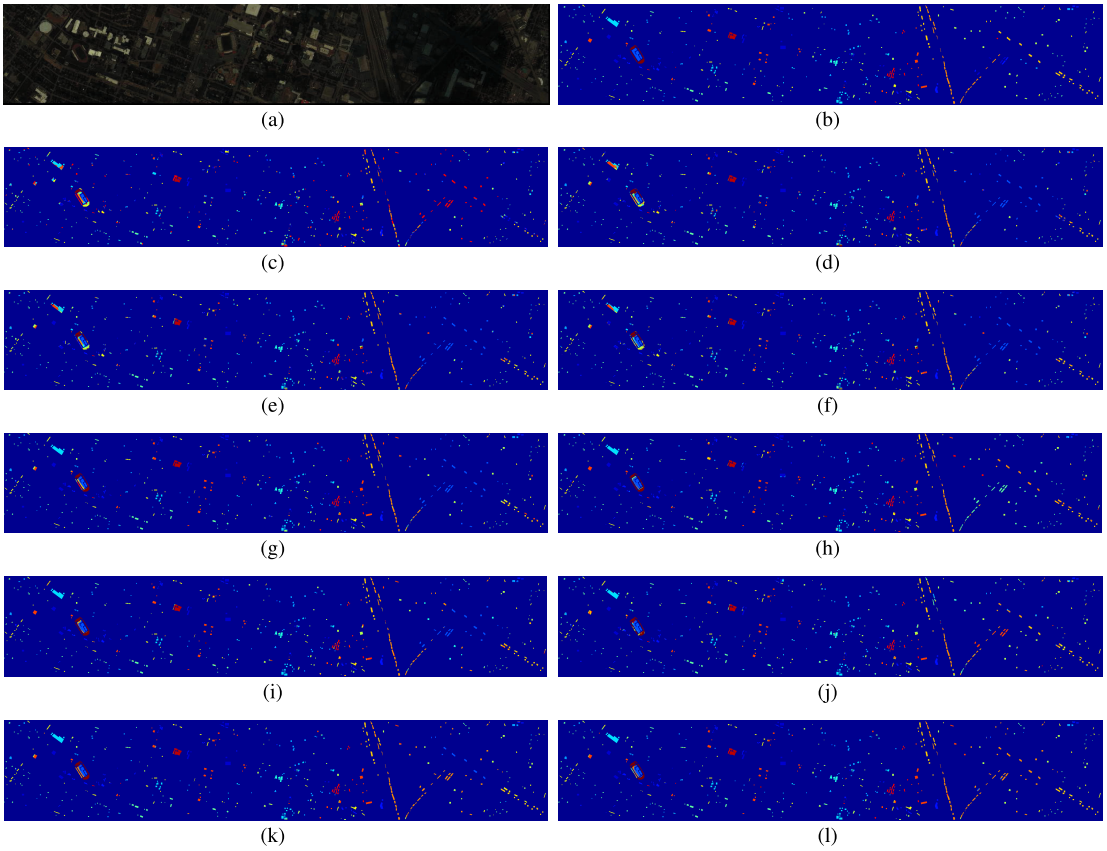


Fig. 12. Classification maps for the Houston dataset. (a) False-color map. (b) Ground-truth map. (c) SVM. (d) 2D-CNN. (e) 2D-TNN-TT. (f) 2D-TNN-TR. (g) 3D-CNN. (h) F-3DCNN. (i) HybridSN. (j) SSTTCL2DNN. (k) 3D-TNN-TT. (l) 3D-TNN-TR.

Pines, Houston, and University of Pavia datasets, respectively. These results also demonstrate the strong generalization and fitting ability of our models.

For the 3D models, they can fully exploit the joint spatial–spectral features of HSIs to achieve superior performance, but the number of parameters is significantly higher than that of the 2D models. Among them, our proposed 3D-TNN-TR model obtains the best classification performance in all three HSI datasets, although it has a slightly larger number of parameters than 3D-TNN-TT. Since the 3D models inherently possess a powerful feature extraction and model fitting capability, the OAs of 3D-TNN-TR on the three datasets are 0.32%, 4.86%, and 1.73% higher than those of 3D-CNN. F-3DCNN adopts a small-sized input to speed up the forward inference of the model, which only extracts the limited spatial context information, yielding the poor results. HybridSN contains five convolutional layers and three FC layers, which offers good classification performance but with the largest number of parameters. SSTTCL2DNN performs the worst in all datasets. Because it contains multiple convolutional

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

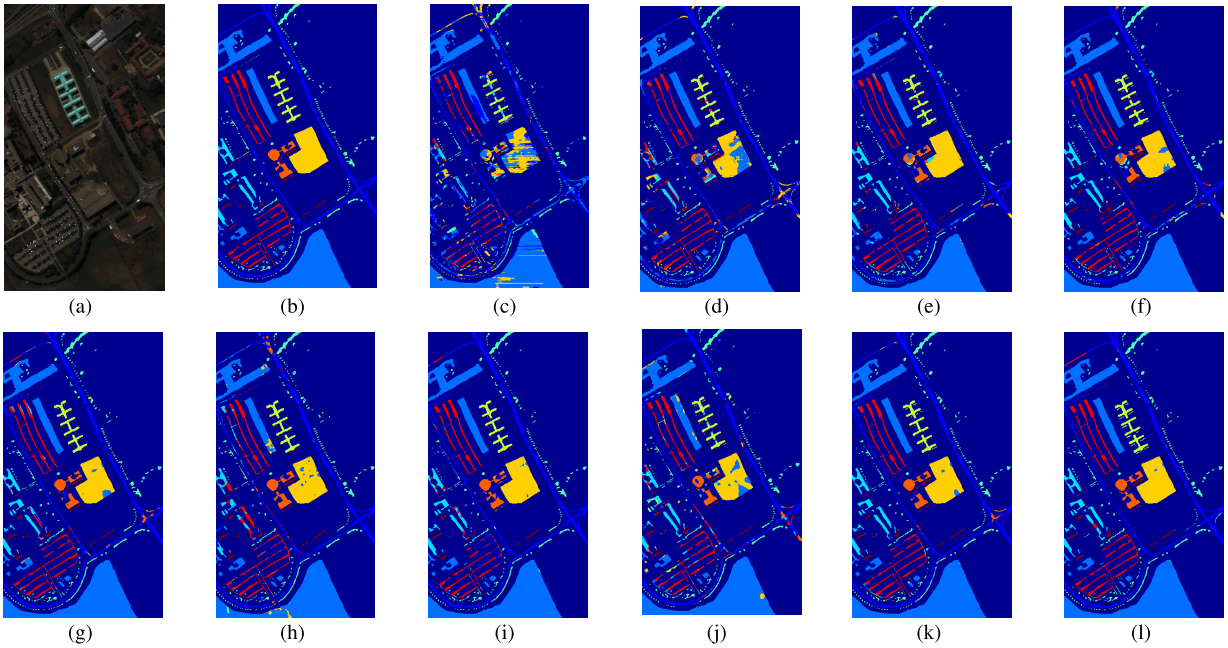


Fig. 13. Classification maps for the University of Pavia dataset. (a) False-color map. (b) Ground-truth map. (c) SVM. (d) 2D-CNN. (e) 2D-TNN-TT. (f) 2D-TNN-TR. (g) 3D-CNN. (h) F-3DCNN. (i) HybridSN. (j) SSTTCL2DNN. (k) 3D-TNN-TT. (l) 3D-TNN-TR.

TABLE VII
CLASSIFICATION RESULTS FOR VERY SMALL TRAINING SETS (TEN SAMPLES FOR EACH CLASS)

Class	Indian Pines				Indian Pines				Indian Pines			
	SVM	2D-CNN	2D-TNN-TT	2D-TNN-TR	SVM	2D-CNN	2D-TNN-TT	2D-TNN-TR	SVM	2D-CNN	2D-TNN-TT	2D-TNN-TR
1	91.67	96.94	**99.72**	**99.72**	63.87	63.72	**77.39**	75.79	38.36	57.03	62.30	**62.39**
2	42.99	31.29	56.90	**59.88**	27.66	37.22	43.73	**45.63**	75.96	82.49	83.59	**84.18**
3	56.51	42.33	64.16	**64.59**	73.78	86.03	92.73	**92.98**	42.20	50.42	64.84	**65.63**
4	73.83	79.69	93.57	**94.01**	42.76	51.43	**61.86**	60.06	66.06	83.15	**83.18**	83.01
5	56.87	48.33	71.46	**71.54**	50.48	66.22	**72.48**	71.79	97.34	96.79	98.80	**99.14**
6	60.42	66.15	79.23	**79.54**	42.99	48.69	60.89	**63.18**	20.97	54.66	**70.60**	68.99
7	97.78	99.44	**100.00**	**100.00**	28.56	31.08	59.86	**61.44**	50.29	48.05	71.55	**73.07**
8	**96.28**	92.20	93.65	94.27	19.15	24.55	27.41	**30.36**	55.66	63.27	78.82	**80.62**
9	**100.00**	**100.00**	**100.00**	**100.00**	32.29	29.42	**53.93**	52.47	81.07	**91.76**	87.47	89.56
10	52.77	50.71	67.62	**69.73**	39.40	47.05	68.48	**68.96**				
11	42.94	41.32	**57.92**	57.80	24.51	53.26	66.29	**67.14**				
12	49.67	43.60	**65.47**	64.87	19.21	21.13	33.46	**34.69**				
13	81.13	95.28	97.79	**97.85**	61.15	82.61	86.91	**88.80**				
14	70.66	80.09	85.49	**86.18**	40.84	75.26	86.53	**86.89**				
15	71.33	76.78	**91.99**	90.85	48.86	55.56	69.55	**70.54**				
16	90.84	98.55	98.07	**98.67**								
OA	56.41	54.36	70.32	**71.03**	38.36	47.35	60.60	**61.06**	59.54	71.68	77.59	**77.96**
AA	70.98	71.42	82.69	**83.09**	41.03	51.55	64.10	**64.71**	58.66	69.74	77.91	**78.51**
κ	48.97	50.60	66.69	**67.49**	33.77	43.39	57.55	**58.04**	46.87	62.91	71.01	**71.46**

layers and a large number of nonlinear functions within its ConvLSTM layer, simply replacing the convolutional layers with the tensorized convolutional layers can reduce the number of parameters but also leads to a performance loss.

Similarly, the classification maps in Figs. 11–13 also support the aforementioned conclusions. It can be clearly seen that our proposed models provide maps closest to the ground-truth maps in all three datasets, which further suggests the validity of the proposed architectures.

D. Classification Performance Under Limited Training Samples

Furthermore, given the expensive cost of obtaining high-quality labels for HSI classification, the performance of all methods is also experimentally evaluated under a very small number of training samples. To do so, a limited number of labeled samples from each class are randomly selected to form the training sets, and additional experiments are performed.

The quantitative comparison results obtained using only ten training samples per class are shown in Tables VII and VIII. It can be seen that our proposed 2D-TNN and 3D-TNN models still outperform the other methods. On the one hand, among 2D classification methods, our 2D-TNN-TR produces the best results on all datasets, improving OA by 16.67%, 13.71%, and 7.85% compared to the 2D-CNN. On the other hand, among 3D classification methods, our 3D-TNN-TR achieves the highest performance on all three datasets, with OA improvements of 6.05%, 2.76%, and 4.53% over the 3D-CNN. Both F-3DCNN and HybridSN with a large number of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
CLASSIFICATION RESULTS FOR VERY SMALL TRAINING SETS (TEN SAMPLES FOR EACH CLASS)

Class	Indian Pines						Houston						University of Pavia					
	3D-CNN	F-3DCNN	HybridSN	SSTTCL2DNN	3D-TNN-TT	3D-TNN-TR	3D-CNN	F-3DCNN	HybridSN	SSTTCL2DNN	3D-TNN-TT	3D-TNN-TR	3D-CNN	F-3DCNN	HybridSN	SSTTCL2DNN	3D-TNN-TT	3D-TNN-TR
1	**100.00**	93.06	98.61	93.89	99.72	99.72	91.62	**92.93**	92.58	76.74	90.60	92.24	68.65	69.51	56.50	53.25	77.39	**78.54**
2	52.54	50.93	47.93	36.69	67.10	**67.46**	83.09	**86.62**	69.48	79.32	83.75	85.25	74.81	70.53	79.64	70.17	81.42	**82.66**
3	54.94	49.59	29.16	51.35	71.94	**72.68**	97.94	89.46	74.23	87.99	97.36	**98.84**	78.94	66.18	36.52	31.66	**86.13**	85.88
4	94.89	64.80	80.57	84.45	**97.36**	97.18	**91.64**	88.43	67.43	71.62	88.37	86.91	**97.40**	86.09	85.06	50.26	89.75	89.97
5	80.19	80.15	76.93	59.09	**84.52**	83.81	99.18	**99.66**	94.75	75.40	99.24	98.74	99.51	99.10	**99.60**	94.16	99.04	98.93
6	**96.03**	91.29	89.58	63.19	92.36	92.63	86.65	79.87	60.73	76.11	**91.21**	91.05	92.28	88.62	72.98	50.10	95.96	**96.43**
7	**100.00**	98.89	**100.00**	**100.00**	**100.00**	**100.00**	78.86	57.40	60.58	61.81	84.29	**87.93**	86.02	85.61	87.49	43.81	93.51	**95.09**
8	**99.91**	94.32	99.14	96.54	98.37	99.04	54.82	54.92	55.12	51.55	60.41	**61.39**	73.01	79.19	57.06	71.29	**95.02**	94.61
9	**100.00**	97.00	**100.00**	**100.00**	**100.00**	**100.00**	73.89	64.89	53.62	63.39	**81.38**	81.11	**95.89**	95.82	89.52	87.02	88.69	88.92
10	69.85	61.52	69.29	76.98	79.56	**80.89**	79.36	53.62	52.62	49.79	86.51	**89.07**						
11	71.60	50.33	52.26	39.46	72.89	**74.53**	75.97	63.09	64.39	57.74	**81.13**	79.74						
12	61.39	47.62	49.38	51.89	**72.86**	71.51	80.51	65.15	54.05	60.49	**78.74**	78.62						
13	**98.62**	98.41	97.59	92.67	98.46	98.51	90.89	69.49	42.27	85.14	**94.36**	94.18						
14	87.95	79.81	66.46	69.78	**88.99**	88.81	97.94	97.85	89.76	95.65	96.75	**98.26**						
15	88.03	60.11	71.65	68.24	**93.11**	91.57	95.84	**98.63**	93.56	87.77	93.14	93.42						
16	**99.04**	98.43	98.55	97.35	98.55	98.31												
OA	74.37	63.59	61.91	57.32	79.93	**80.42**	83.24	75.36	67.80	68.62	85.38	**86.00**	79.15	76.06	72.69	62.28	85.58	**86.38**
AA	84.69	76.01	76.69	73.85	88.49	**88.54**	85.21	77.47	68.34	72.03	87.14	**87.71**	85.17	82.29	73.82	61.30	89.66	**90.12**
κ	70.98	59.21	57.44	52.69	77.41	**77.94**	81.89	73.41	65.25	66.15	84.20	**84.87**	73.79	69.89	64.73	51.88	81.73	**82.64**

model parameters perform relatively poorly in the experiments under small training sets, whose OAs are much lower than that of our proposed models. SSTTCL2DNN achieves the worst classification accuracy because it has many nonlinear activation functions, leading to easy overfitting of the model. From the above comparison results, it can be seen that the improvements in classification performance obtained by our proposed models with small training sets is substantially greater than that obtained with standard training sets, mostly due to the fact that overfitting occurs more severely under small training sets. Thus, our proposed models exhibit great representational ability and can learn better features to gain higher classification accuracy in complex training scenarios.

Besides the experiments with small training sets described above, 20, 30, and 40 label samples from each of these three HSI datasets are also randomly selected for training to further validate the reliability of the proposed models. Due to the insufficiency of the total number of samples, only ten samples from classes 7 and 9 are utilized in the Indian Pines dataset. The OA curves of all models under different numbers of training samples are shown in Fig. 14, which shows that (as expected) the classification performance of all methods increases with the number of samples. However, the proposed 2D-TNN-TT and 2D-TNN-TR models are clearly superior to 2D-CNN with small training samples and can even approach the performance of 3D models on the Indian Pines dataset. The 3D-TNN-TR performs well on the three considered HSI datasets and is superior to the others. The above experiments verify the reliability of the proposed TNN models, which can commendably extract highly discriminative spatial or spatial–spectral features using different numbers of training samples, leading to better classification results in all cases.

E. Comparing Number of Parameters and Testing Time

In this section, a quantitative analysis of the parameters needed by all considered models is presented. Table IX compares the number of parameters of our proposed TNNs and other deep models. It can be seen that, both in the 2D and 3D models, the number of parameters of our proposed TNNs is much smaller than that of other considered deep models.

TABLE IX
NUMBER OF PARAMETERS NEEDED BY THE CONSIDERED MODELS

Model	Indian Pines	Houston	University of Pavia
2D-CNN	95, 184	95, 055	94, 281
2D-TNN-TT	24, 631	22, 301	22, 224
2D-TNN-TR	24, 080	20, 063	21, 673
3D-CNN	279, 856	279, 727	278, 953
F-3DCNN	995, 456	995, 327	994, 553
HybridSN	5, 122, 176	612, 351	4, 844, 794
SSTTCL2DNN	454, 208	125, 952	453, 760
3D-TNN-TT	28, 387	23, 417	25, 980
3D-TNN-TR	29, 981	18, 534	27, 574

The fact that our TNN models require a small number of parameters is mainly caused by the tensorized convolutional layer, TT-layer, and GAP layer. Among 2D models, 2D-TNN-TR needs the lowest number of parameters on all three datasets. To be specific, compared with 2D-CNN, 2D-TNN-TR achieves 3.95×, 4.74×, and 4.35× CR on the Indian Pines, Houston, and University of Pavia datasets, respectively. In 3D models, F-3DCNN, HybridSN, and SSTTCL2DNN require a large number of parameters, mainly because they contain multiple large-sized FC layers. Thus, the GAP layer is used instead of the largest FC layer in the 3D-CNN, so the number of parameters is smaller. On the other hand, due to the large amount of redundancy in 3D models, 3D-TNN-TR obtains a CR of up to 9.33×, 15.09×, and 10.12× on the three datasets, which is significantly better than that of the 2D models. Although 3D-TNN-TT involves with less parameters than 3D-TNN-TR on the Indian Pines and University of Pavia datasets, the difference between their performance is insignificant. Overall, our newly proposed TNN models need less parameters but gain superior classification accuracy.

Moreover, the testing time together with the size of sample needed by all considered models is listed in Table X to analyze the computational complexity. As can be seen from Table X, the proposed TNN models take longer testing time than the original 2D- and 3D-CNNs, though require less storage space by model compression. The possible reasons are that, on the one hand, the sizes of convolution kernels

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

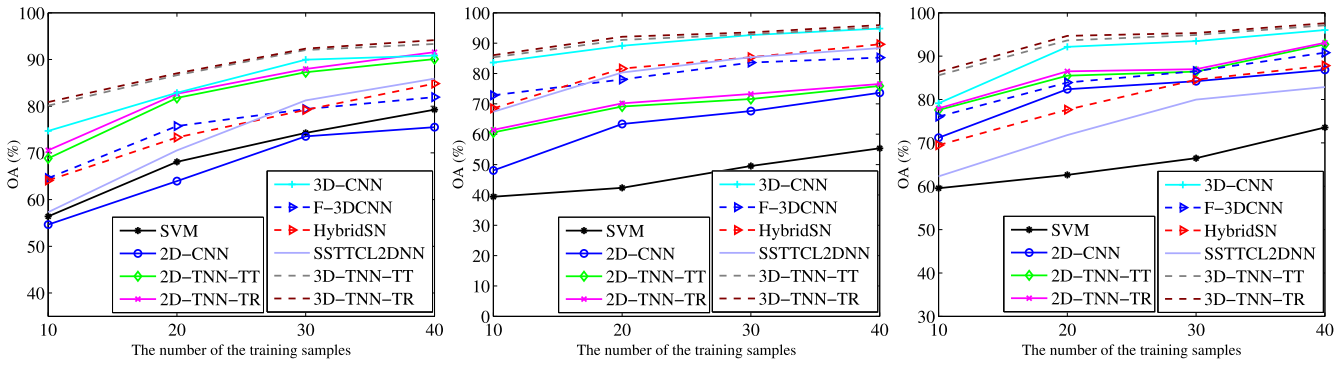


Fig. 14. OA obtained by using different numbers of training samples for three HSI datasets. (a) Indian Pines. (b) Houston. (c) University of Pavia.

TABLE X
TESTING TIME (IN SECONDS) AND THE SIZE OF SAMPLE NEEDED BY THE CONSIDERED MODELS

Model	Indian Pines		Houston		University of Pavia	
	Size of Sample	Testing	Size of Sample	Testing	Size of Sample	Testing
2D-CNN	$17\times17\times1$	0.22	$17\times17\times1$	0.29	$17\times17\times1$	1.00
2D-TNN-TT	$27\times27\times1$	0.38	$17\times17\times1$	0.38	$21\times21\times1$	1.34
2D-TNN-TR	$27\times27\times1$	1.09	$17\times17\times1$	0.86	$21\times21\times1$	3.62
3D-CNN	$17\times17\times10$	0.34	$17\times17\times10$	0.47	$17\times17\times10$	1.57
F-3DCNN	$11\times11\times20$	0.24	$11\times11\times20$	0.31	$11\times11\times20$	1.06
HybridSN	$25\times25\times30$	1.81	$13\times13\times20$	0.35	$25\times25\times15$	2.71
SSTTCL2DNN	$27\times27\times5$	1.82	$11\times11\times10$	2.77	$27\times27\times5$	9.69
3D-TNN-TT	$27\times27\times10$	1.37	$17\times17\times10$	0.83	$21\times21\times10$	4.04
3D-TNN-TR	$27\times27\times10$	3.58	$17\times17\times10$	2.11	$21\times21\times10$	9.93

and input samples are enlarged to extract more spatial–spectral information; on the other hand, the tensorized convolutional layers contain some additional matrix/tensor multiplications, besides extra transposition and permutation operations. For other considered models, F-3DCNN and HybridSN have the similar network structure, where the former adopts small-sized input samples, speeding up the model but degrading classification performance, while the latter employs larger sized input samples, resulting in longer testing time. As a variant of RNNs, SSTTCL2DNN needs to complete the sequence operations serially and increases the computational cost, especially with the raise of PCs number in the input samples. Furthermore, the proposed TT-based models provide appropriate time for testing. In contrast, the testing time of TR-based models with the decomposition of a more complex ring structure is a little longer, but it achieves better performance. To sum up, our compressed models can provide promising classification performance with the acceptable testing time.

VI. CONCLUSION

In this article, two TNN models, i.e., 2D-TNN-TT/TR and 3D-TNN-TT/TR, for HSI classification have been proposed, based on different tensor decomposition methods. The proposed models, which consist of tensorized convolutional layers (using chain calculation) and tensorized FC layers, require a smaller number of parameters than other deep models. A series of quantitative experiments on three widely used HSI datasets demonstrates that the proposed models can offer the state-of-the-art performance with less parameters. In particular, low-rank tensor decomposition is used to reduce the number of parameters of the proposed models, making them less prone to overfitting on HSI datasets with small training sets, thereby improving their classification performance. It is worth mentioning that the proposed tensorized convolutional layers based on chain calculation are universal and can be used to achieve effective compression of other HSI classification models under the premise of low complexity.

ACKNOWLEDGMENT

The authors would like to thank the Editor-in-Chief, Associate Editor, and anonymous reviewers for their valuable comments and suggestions.

REFERENCES

[1] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "Deep learning classifiers for hyperspectral imaging: A review," *ISPRS J. Photogramm. Remote Sens.*, vol. 158, pp. 279–317, Dec. 2019.

[2] J. M. Bioucas-Dias, A. Plaza, G. Camps-Valls, P. Scheunders, N. Nasrabadi, and J. Chanussot, "Hyperspectral remote sensing data analysis and future challenges," *IEEE Geosci. Remote Sens. Mag.*, vol. 1, no. 2, pp. 6–36, Jun. 2013.

[3] M. D. Raya-Sereno et al., "Residual effect and N fertilizer rate detection by high-resolution VNIR-SWIR hyperspectral imagery and solar-induced chlorophyll fluorescence in wheat," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–17, 2022.

[4] X. Jiang, J. Zhen, J. Miao, D. Zhao, J. Wang, and S. Jia, "Assessing mangrove leaf traits under different pest and disease severity with hyperspectral imaging spectroscopy," *Ecol. Indicators*, vol. 129, Oct. 2021, Art. no. 107901.

[5] J. Cai, J. Chen, X. Dou, and Q. Xing, "Using machine learning algorithms with in situ hyperspectral reflectance data to assess comprehensive water quality of urban rivers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–13, 2022.

[6] I. C. C. Acosta, M. Khodadadzadeh, R. Tolosana-Delgado, and R. Gloaguen, "Drill-core hyperspectral and geochemical data integration in a superpixel-based machine learning framework," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 4214–4228, 2020.

[7] R. Hänsch and O. Hellwich, "Fusion of multispectral LiDAR, hyperspectral, and RGB data for urban land cover classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 18, no. 2, pp. 366–370, Feb. 2021.

[8] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[9] Y. Chen, N. M. Nasrabadi, and T. D. Tran, "Hyperspectral image classification using dictionary-based sparse representation," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 10, pp. 3973–3985, Oct. 2011.

[10] J. Ham, Y. Chen, M. M. Crawford, and J. Ghosh, "Investigation of the random forest framework for classification of hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 492–501, Mar. 2005.

[11] P. Zhong, P. Zhang, and R. Wang, "Dynamic learning of SMLR for feature selection and classification of hyperspectral data," *IEEE Geosci. Remote Sens. Lett.*, vol. 5, no. 2, pp. 280–284, Apr. 2008.

[12] G. Camps-Valls, L. Gomez-Chova, J. Munoz-Mari, J. Vila-Frances, and J. Calpe-Maravilla, "Composite kernels for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 3, no. 1, pp. 93–97, Jan. 2006.

[13] J. Peng and Q. Du, "Robust joint sparse representation based on maximum correntropy criterion for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 12, pp. 7152–7164, Dec. 2017.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

[14] S. Yang et al., "Integration of crop growth model and random forest for winter wheat yield estimation from UAV hyperspectral imagery," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 6253–6269, 2021.

[15] X. Wang, "Kronecker factorization-based multinomial logistic regression for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[16] J. Lanchantin, T. Wang, V. Ordonez, and Y. Qi, "General multi-label image classification with transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Nashville, TN, USA, Jun. 2021, pp. 16473–16483.

[17] Z. Cui, X. Wang, N. Liu, Z. Cao, and J. Yang, "Ship detection in large-scale SAR images via spatial shuffle-group enhance attention," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 379–391, Jan. 2021.

[18] P. Voigtlaender, J. Luiten, P. Torr, and B. Leibe, "Siam R-CNN: Visual tracking by re-detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, Jun. 2020, pp. 6577–6587.

[19] Y. Ye, P. Huang, and Y. Zhang, "Deep learning-based fault diagnostic network of high-speed train secondary suspension systems for immunity to track irregularities and wheel wear," *Railway Eng. Sci.*, vol. 30, no. 1, pp. 96–116, Mar. 2022.

[20] W. Hu, Y. Huang, W. Li, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, pp. 1–12, Jan. 2015.

[21] J.-Y. Yang, H.-C. Li, W.-S. Hu, L. Pan, and Q. Du, "Adaptive cross-attention-driven spatial–spectral graph convolutional network for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[22] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[23] T. Li, J. Zhang, and Y. Zhang, "Classification of hyperspectral image based on deep belief networks," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Paris, France, Oct. 2014, pp. 5132–5136.

[24] K. Makantasis, K. Karantzalos, A. Doulamis, and N. Doulamis, "Deep supervised learning for hyperspectral data classification through convolutional neural networks," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Milan, Italy, Jul. 2015, pp. 4959–4962.

[25] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[26] Y. Li, H. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, Jan. 2017.

[27] S.-H. Gao, M.-M. Cheng, K. Zhao, X.-Y. Zhang, M.-H. Yang, and P. Torr, "Res2Net: A new multi-scale backbone architecture," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 2, pp. 652–662, Feb. 2021.

[28] N. Takahashi and Y. Mitsufuji, "Densely connected multidilated convolutional networks for dense prediction tasks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Nashville, TN, USA, Jun. 2021, pp. 993–1002.

[29] W.-Y. Wang, H.-C. Li, Y.-J. Deng, L.-Y. Shao, X.-Q. Lu, and Q. Du, "Generative adversarial capsule network with ConvLSTM for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 18, no. 3, pp. 523–527, Mar. 2021.

[30] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[31] W. Wang, S. Dou, Z. Jiang, and L. Sun, "A fast dense spectral–spatial convolution network framework for hyperspectral images classification," *Remote Sens.*, vol. 10, no. 7, p. 1068, Jul. 2018.

[32] M. E. Paoletti et al., "Capsule networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2145–2160, Apr. 2019.

[33] X. Yang, W. Cao, Y. Lu, and Y. Zhou, "Hyperspectral image transformer classification networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2022.

[34] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "CNN-enhanced graph convolutional network with pixel- and superpixel-level feature fusion for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8657–8671, Oct. 2021.

[35] T.-L. Chin, K.-Y. Chen, D.-Y. Chen, and D.-E. Lin, "Intelligent real-time earthquake detection by recurrent neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 8, pp. 5440–5449, Aug. 2020.

[36] J. Du, C.-M. Vong, and C. L. P. Chen, "Novel efficient RNN and LSTM-like architectures: Recurrent and gated broad learning systems and their applications for text classification," *IEEE Trans. Cybern.*, vol. 51, no. 3, pp. 1586–1597, Mar. 2021.

[37] H.-C. Li, W.-S. Hu, W. Li, J. Li, Q. Du, and A. Plaza, "A^3 CLNN: Spatial, spectral and multiscale attention ConvLSTM neural network for multisource remote sensing data classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 2, pp. 747–761, Feb. 2022.

[38] W.-S. Hu, H.-C. Li, R. Wang, F. Gao, Q. Du, and A. Plaza, "Pseudo complex-valued deformable ConvLSTM neural network with mutual attention learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–17, 2022.

[39] L. Mou, P. Ghamisi, and X. X. Zhu, "Deep recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3639–3655, Jul. 2017.

[40] Q. Liu, F. Zhou, R. Hang, and X. Yuan, "Bidirectional-convolutional LSTM based spectral-spatial feature learning for hyperspectral image classification," *Remote Sens.*, vol. 9, no. 12, p. 1330, Dec. 2017.

[41] W.-S. Hu, H.-C. Li, L. Pan, W. Li, R. Tao, and Q. Du, "Spatial–spectral feature extraction via deep ConvLSTM neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 6, pp. 4237–4250, Jun. 2020.

[42] Y. Cheng, D. Wang, P. Zhou, and T. Zhang, "Model compression and acceleration for deep neural networks: The principles, progress, and challenges," *IEEE Signal Process. Mag.*, vol. 35, no. 1, pp. 126–136, Jan. 2018.

[43] P. Maji and R. Mullins, "On the reduction of computational complexity of deep convolutional neural networks," *Entropy*, vol. 20, no. 4, p. 305, Apr. 2018.

[44] A. Howard et al., "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Seoul, South Korea, Oct. 2019, pp. 1314–1324.

[45] M. Lin et al., "Filter sketch for network pruning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 12, pp. 7091–7100, Dec. 2022.

[46] Z. Tang, X. Peng, K. Li, and D. N. Metaxas, "Towards efficient U-Nets: A coupled and quantized approach," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 8, pp. 2038–2050, Aug. 2020.

[47] S. Chen, R. Zhan, W. Wang, and J. Zhang, "Learning slimming SAR ship object detector through network pruning and knowledge distillation," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 1267–1282, 2021.

[48] W. Sun, S. Chen, L. Huang, H. C. So, and M. Xie, "Deep convolutional neural network compression via coupled tensor decomposition," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 3, pp. 603–616, Apr. 2021.

[49] D. O'Neill, B. Xue, and M. Zhang, "Evolutionary neural architecture search for high-dimensional skip-connection structures on DenseNet style networks," *IEEE Trans. Evol. Comput.*, vol. 25, no. 6, pp. 1118–1132, Dec. 2021.

[50] L. Fang, G. Liu, S. Li, P. Ghamisi, and J. A. Benediktsson, "Hyperspectral image classification with squeeze multibias network," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 3, pp. 1291–1301, Mar. 2019.

[51] Z. Meng, L. Jiao, M. Liang, and F. Zhao, "A lightweight spectral-spatial convolution module for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[52] Y. Cui, J. Xia, Z. Wang, S. Gao, and L. Wang, "Lightweight spectral–spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–14, 2022.

[53] R. Li and C. Duan, "LiteDenseNet: A lightweight network for hyperspectral image classification," 2020, *arXiv:2004.08112*.

[54] B. Cui, X.-M. Dong, Q. Zhan, J. Peng, and W. Sun, "LiteDepthwiseNet: A lightweight network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2022.

[55] S. Qiao, X.-M. Dong, J. Peng, and W. Sun, "LiteSCANet: An efficient lightweight network based on spectral and channel-wise attention for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 11655–11668, 2021.

[56] B. Liu, X. Yu, P. Zhang, A. Yu, Q. Fu, and X. Wei, "Supervised deep feature extraction for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 4, pp. 1909–1921, Apr. 2018.

[57] X. Cao, M. Ren, J. Zhao, H. Li, and L. Jiao, "Hyperspectral imagery classification based on compressed convolutional neural network," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 9, pp. 1583–1587, Sep. 2020.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.

[58] Y. Chen, K. Zhu, L. Zhu, X. He, P. Ghamisi, and J. A. Benediktsson, "Automatic design of convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 7048–7066, Sep. 2019.
[59] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.
[60] A. Novikov, D. Podoprikhin, A. Osokin, and D. Vetrov, "Tensorizing neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, Montreal, QC, Canada, 2015, pp. 442–450.
[61] J. Kossaifi, Z. C. Lipton, A. Kolbeinsson, A. Khanna, T. Furlanello, and A. Anandkumar, "Tensor regression networks," *J. Mach. Learn. Res.*, vol. 21, no. 1, pp. 4862–4882, Jan. 2020.
[62] T. Garipov, D. Podoprikhin, A. Novikov, and D. Vetrov, "Ultimate tensorization: Compressing convolutional and FC layers alike," 2016, *arXiv:1611.03214*.
[63] J. Su, J. Li, B. Bhattacharjee, and F. Huang, "Tensorial neural networks: Generalization of neural networks and application to model compression," 2018, *arXiv:1805.10352*.
[64] D. Wang, G. Zhao, G. Li, L. Deng, and Y. Wu, "Compressing 3DCNNs based on tensor train decomposition," *Neural Netw.*, vol. 131, pp. 215–230, Nov. 2020.
[65] I. V. Oseledets, "Tensor-train decomposition," *SIAM J. Sci. Comput.*, vol. 33, no. 5, pp. 2295–2317, Jan. 2011.
[66] R. Selvan and E. B. Dam, "Tensor networks for medical image classification," in *Proc. Int. Conf. Med. Imag. Deep Learn.*, Montreal, QC, Canada, 2020, pp. 721–732.
[67] X. Jin et al., "CTNN: A convolutional tensor-train neural network for multi-task brainprint recognition," *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 29, pp. 103–112, 2021.
[68] B. Zhao, X. Li, and X. Lu, "TTH-RNN: Tensor-train hierarchical recurrent neural network for video summarization," *IEEE Trans. Ind. Electron.*, vol. 68, no. 4, pp. 3629–3637, Apr. 2021.
[69] C. Dai et al., "Video scene segmentation using tensor-train faster-RCNN for multimedia IoT systems," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9697–9705, Jun. 2021.
[70] K. Makantasis, A. D. Doulamis, N. D. Doulamis, and A. Nikitakis, "Tensor-based classification models for hyperspectral data analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 12, pp. 6884–6898, Dec. 2018.
[71] W.-S. Hu, H.-C. Li, Y.-J. Deng, X. Sun, Q. Du, and A. Plaza, "Lightweight tensor attention-driven ConvLSTM neural network for hyperspectral image classification," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 3, pp. 734–745, Apr. 2021.
[72] Q. Zhao, M. Sugiyama, L. Yuan, and A. Cichocki, "Learning efficient tensor representations with ring-structured networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Brighton, U.K., May 2019, pp. 8608–8612.
[73] C.-C. Chang and C.-J. Lin, "LIBSVM: A library for support vector machines," *ACM Trans. Intell. Syst. Technol.*, vol. 2, no. 3, pp. 1–27, Apr. 2011.
[74] B. Rasti et al., "Feature extraction for hyperspectral imagery: The evolution from shallow to deep: Overview and toolbox," *IEEE Geosci. Remote Sens. Mag.*, vol. 8, no. 4, pp. 60–88, Dec. 2020.
[75] M. Ahmad, A. M. Khan, M. Mazzara, S. Distefano, M. Ali, and M. S. Sarfraz, "A fast and compact 3-D CNN for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.



Tian-Yu Ma received the B.Sc. degree in communication engineering from Southwest Jiaotong University, Chengdu, China, in 2018, where he is currently pursuing the Ph.D. degree in traffic information engineering and control with the School of Information Science and Technology.

His research interests include tensor decomposition, neural network compression, and hyperspectral image processing.



Heng-Chao Li (Senior Member, IEEE) received the B.Sc. and M.Sc. degrees from Southwest Jiaotong University, Chengdu, China, in 2001 and 2004, respectively, and the Ph.D. degree from the Graduate University of Chinese Academy of Sciences, Beijing, China, in 2008, all in information and communication engineering.

He is currently a Full Professor with the School of Information Science and Technology, Southwest Jiaotong University. From November 2013 to October 2014, he has been a Visiting Scholar working with Prof. William J. Emery at the University of Colorado Boulder, Boulder, CO, USA. His research interests include statistical analysis of synthetic aperture radar (SAR) images, remote sensing image processing, and pattern recognition.



Rui Wang received the Ph.D. degree in information and communication engineering from Southwest Jiaotong University, Chengdu, China, in 2017.

From September 2015 to September 2016, she was a Visiting Scholar with Ghent University, Ghent, Belgium. She is currently an Associate Professor with the Sichuan Provincial Key Laboratory of Information Coding and Transmission, Southwest Jiaotong University. Her research interests include remote sensing image processing.



Qian Du (Fellow, IEEE) received the Ph.D. degree in electrical engineering from the University of Maryland, Baltimore, MD, USA, in 2000.

She is currently a Bobby Shackouls Professor with the Department of Electrical and Computer Engineering, Mississippi State University, Starkville, MS, USA. Her research interests include hyperspectral remote sensing image analysis and applications, pattern classification, data compression, and neural networks.



Xiuping Jia (Fellow, IEEE) received the B.Eng. degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 1982, and the Ph.D. degree in electrical engineering from the University of New South Wales, Canberra, ACT, Australia, in 1996.

She is currently an Associate Professor with the School of Engineering and Information Technology, University of New South Wales. She has coauthored the remote sensing textbook titled *Remote Sensing Digital Image Analysis* (Springer-Verlag, Third Edition, 1999, and Fourth Edition, 2006). Her research interests include remote sensing, hyperspectral image processing, and spatial data analysis.

Dr. Jia is a Subject Editor of the *Journal of Soils and Sediments* and the Associate Editor-in-Chief of the IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING.



Antonio Plaza (Fellow, IEEE) received the M.Sc. and Ph.D. degrees in computer engineering from the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, University of Extremadura, Cáceres, Spain, in 1999 and 2002, respectively.

He is currently the Head of the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, University of Extremadura. His research interests include hyperspectral data processing and parallel computing of remote sensing data.

Dr. Plaza has served as the Editor-in-Chief for the IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING from 2013 to 2017 and has guest-edited ten special issues on hyperspectral remote sensing for different journals.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 09:48:41 UTC from IEEE Xplore. Restrictions apply.